

ABSTRACT

In the contemporary era, approximately 70% of user data is readily available online, stored in centralized servers where the potential for misuse or alteration by owners or database managers poses a significant threat. Users often find themselves compelled to disclose sensitive information, including address, zip code, gender, and medical history, for identity verification or Know Your Customer (KYC) registrations. The lack of a direct mechanism for users to ascertain the security of their data on these servers raises concerns about privacy breaches and unauthorized use of personal information.

To mitigate the risks associated with the leakage of sensitive information, we propose the implementation of Blockchain-based KYC Sharing. Blockchain technology, renowned for its inherent support for data encryption, access control, and data immutability, presents a robust solution. Each record in the Blockchain is treated as a transaction or block, uniquely associated with a HASHCODE. During the addition of new records, Blockchain meticulously verifies the HASHCODE of all preceding records. If the data remains unaltered, the verification process is successful, rendering the Blockchain immutable. Access to data stored in the Blockchain is restricted to users with explicit permissions, ensuring secure sharing of records.

In our proposed system, when any company or organization accesses a user's KYC data, Blockchain automatically triggers an EMAIL notification to the KYC user, providing transparency and accountability in the access process.

The inherent advantages of Blockchain, including data integrity and secure sharing capabilities, have led us to adopt it as the primary platform for managing all KYC records. Smart Contracts, designed using Solidity code, facilitate the storage and retrieval of data in the Blockchain. These Smart Contracts contain functions that govern how user data is managed within the Blockchain, ensuring a secure and transparent KYC sharing environment. The proposed Blockchain-based KYC Sharing system addresses the pressing concerns associated with centralized storage and sets the stage for a more secure, accountable, and user-centric approach to sensitive information management.

1.INTRODUCTION

In our increasingly digitized world, the proliferation of online platforms has resulted in a substantial volume of user data, particularly through identity verification processes like Know Your Customer (KYC). However, the conventional methods of storing and sharing identity information often fall short in terms of security, leaving users vulnerable to data breaches and misuse. The "Secure Sharing of Identity (KYC)" project aims to address these concerns by leveraging advanced technologies to establish a robust and trustworthy framework for managing and sharing sensitive identity information. The rapid growth of digital services has led to an increased demand for secure and efficient identity verification systems. Traditional KYC (Know Your Customer) processes are largely paper-based or centralized digital systems that expose users to privacy and security risks. Data breaches, unauthorized access, and redundancy in identity submissions have been recurring issues. In response, this project explores a blockchain-based decentralized identity verification solution that offers transparency, control, and tamper-resistance for user data.

In today's world, individuals are required to complete KYC procedures for banking, insurance, telecom services, and even cryptocurrency platforms. This growing reliance on KYC has made it essential to develop systems that not only streamline the process but also protect user information. A secure and reliable identity system forms the foundation of trust in online services, and this project targets the transformation of that foundation using cutting-edge blockchain solutions.

1.1 Domain Description

This project falls under the domain of digital identity management with a focus on Know Your Customer (KYC) procedures. KYC is a critical process used by financial institutions and service providers to verify the identity of their customers. Digital identity management aims to provide users with control over their personal data while enabling institutions to validate identity with high assurance and low friction. By integrating blockchain technology, the domain evolves to support decentralized, immutable, and transparent identity systems.

Blockchain technology adds a layer of integrity and transparency that traditional systems lack. This domain benefits from immutable records, secure access control, and programmable rules through smart contracts, enabling a user-centric and efficient approach to identity verification.

The proposed project operates at the intersection of cybersecurity, cryptography, and regulatory technology, aligning with modern digital transformation needs.

1.2 Problem Definition

Traditional KYC systems rely heavily on centralized servers and manual processes, making them vulnerable to data breaches, manipulation, and unauthorized access. Users must repeatedly share their sensitive personal documents with multiple entities, increasing the risk of data leakage. There is also little transparency or control for users to monitor who accesses their information. These limitations lead to inefficiencies, poor user trust, and regulatory compliance challenges.

Additionally, current KYC systems incur high administrative costs and time delays due to repetitive document submissions and verification steps across organizations. Regulatory bodies often face difficulties in auditing or tracing access and verification trails. This results in a fragmented ecosystem lacking interoperability, user control, and auditability—three pillars essential to a trusted identity infrastructure.

1.3 Proposed Solution

The proposed system introduces a decentralized KYC platform built on blockchain technology. Each user's identity is securely stored as encrypted data linked to blockchain transactions. Smart contracts are used to manage user consent and automate access permissions. When a verified entity attempts to access a user's KYC data, the system checks access rights and notifies the user via email. This approach ensures transparency, non-repudiation, and control over personal information, addressing key issues of traditional systems.

The architecture integrates Ethereum-based smart contracts, the Interplanetary File System (IPFS) for document storage, and Python for backend APIs. These components collectively enable data immutability, distributed storage, and efficient interaction between users and verifiers. The platform is scalable, low-cost, and adaptable for integration with third-party verification systems.

1.4 Objective

The primary objective of this project is to develop a blockchain-based KYC system that allows secure, efficient, and user-controlled sharing of identity information. The system aims to eliminate redundant identity verification steps, minimize privacy risks, and improve transparency. By leveraging smart contracts and immutable data storage, the solution ensures that only authorized parties can access KYC data with user awareness and consent.

Key goals include:

- Enhancing user privacy through cryptographic encryption and controlled access
- Enabling real-time audit trails for every access event
- Reducing operational costs for organizations by eliminating repeated verifications
- Empowering users with full control over their data visibility

The project envisions a next-generation digital KYC process that aligns with modern regulatory standards and fosters user trust in digital interactions.

2. LITERATURE SURVEY

The literature survey focuses on the current state of identity verification systems, the evolution of KYC procedures, and the integration of blockchain technology into secure identity management. It examines previous research and technological solutions proposed to address the challenges of centralization, lack of transparency, and data insecurity in KYC systems. This review is essential to understand the limitations of existing solutions and how the proposed system can bridge the gaps.

2.1 Existing System

Currently, the majority of identity data is stored in centralized systems, posing inherent risks of unauthorized access, manipulation, and potential breaches. Users, especially during KYC procedures, are often required to disclose sensitive information, including personal details and documentation, without adequate assurance of its security. This reliance on centralized servers lacking advanced security features exposes users to the ever-looming threat of identity theft, privacy invasion, and misuse. Traditional KYC systems are typically centralized, with financial institutions and service providers maintaining their own private databases for customer identification and verification. These systems store sensitive personal documents such as government IDs, utility bills, and photographs. Because they are stored on central servers,

these databases are vulnerable to hacking, unauthorized modifications, and insider threats. Additionally, every organization requiring KYC must independently verify and store user data, resulting in redundancy, inefficiencies, and increased cost.

The current systems also lack interoperability. If a user completes a KYC process with one organization, they must repeat the process entirely with another. This increases customer friction and operational overhead. Moreover, there is minimal auditability in how user data is accessed and shared, reducing trust and making regulatory compliance harder.

2.2Proposed System

To enhance the security and privacy of identity information, the proposed "Secure Sharing of Identity (KYC)" project introduces a state-of-the-art solution by incorporating blockchain technology. Blockchain, renowned for its immutable and decentralized nature, ensures data integrity, access control, and encryption. Each identity record becomes a secure transaction within the blockchain, associated with a unique identifier. The system employs smart contracts to facilitate secure data sharing, allowing access only to authorized parties while maintaining a transparent and traceable record of data interactions.

In addition to blockchain, advanced notification mechanisms are implemented. When any entity requests access to a user's identity information, the system triggers instant notifications to the user, enhancing transparency and empowering users with real-time awareness and control over their data. The proposed system leverages blockchain technology to establish a decentralized and secure method for storing and sharing KYC data. Instead of a central database, user identities are stored as encrypted data linked to blockchain transactions. Blockchain's immutability ensures data cannot be tampered with once entered. Access to KYC records is governed by smart contracts, which enforce permissions and log every access attempt transparently.

Organizations can retrieve KYC information only with the user's consent. Notifications are triggered whenever access is granted, ensuring user awareness. The system also supports document storage through IPFS, which provides decentralized file storage. This approach eliminates duplication, enhances security, reduces cost, and builds user trust.

2.3 Advantages of proposed system

- **Data Integrity:** Blockchain guarantees that once user data is recorded, it cannot be altered without detection.
- **Transparency and Auditability:** All access attempts are logged and verifiable by both the user and the organization.
- **Reduced Redundancy:** Once a user's KYC is stored on the blockchain, it can be reused with permission, reducing repetitive submissions.
- **User Consent:** Access to user data is permission-based and confirmed with user notifications.
- **Cost Efficiency:** The decentralized approach reduces storage costs and eliminates the need for repeated verification.
- **Security:** Encryption, decentralized storage, and smart contracts minimize data breaches and internal misuse.

2.4 Feasibility Study

This section assesses whether the proposed system can be realistically developed and deployed with the available resources. The analysis covers operational, technical, and economic aspects. Operationally, the system addresses a real-world need for a secure and efficient KYC process. It aligns with trends in financial technology and regulatory compliance.

The availability of open-source blockchain platforms, libraries for smart contracts, and backend development tools confirms that there are sufficient technical resources to implement the system. Additionally, the cost of infrastructure is minimal due to the use of freely available software and cloud-based decentralized services.

2.5. Technical Feasibility

The proposed system is technically feasible because it utilizes widely supported tools and frameworks:

- **Blockchain (Ethereum):** For immutable record keeping and smart contract execution.

- IPFS: For decentralized and secure file storage.
- Solidity: For writing smart contracts.
- Python (Flask/Django): For backend development.

These technologies are robust, community-supported, and suitable for building scalable and secure applications. The existing developer ecosystem ensures adequate support and documentation for integration. This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

2.6. Survey of Technology

A variety of technologies underpin the proposed system:

- Blockchain: Provides an immutable ledger to store transactions related to identity access.
- IPFS (Interplanetary File System): Ensures distributed storage of large identity documents.
- Smart Contracts (Solidity): Automate permission checking and access control.
- Python: Facilitates server-side logic and web APIs.
- Ganache & Truffle Suite: Tools for local Ethereum development and testing.
- Web3.js / Web3.py: Libraries to interact with Ethereum from frontend and backend.

These technologies provide a seamless platform for secure and decentralized KYC management. The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

2.7 Feasibility of Technology

The chosen technologies are scalable, widely adopted, and battle-tested. Ethereum and IPFS are already used in high-stakes applications like finance, healthcare, and supply chain management. The modular architecture allows the project to evolve with emerging tools and libraries.

Furthermore, Python's simplicity and integration capabilities make it ideal for building flexible backend services that interact with blockchain and file systems. The feasibility is reinforced by real-world use cases demonstrating the success of similar architectures in decentralized identity systems.

2.8 Economic Feasibility

The economic feasibility revolves around the project's cost-effectiveness. Since the core technologies—Ethereum test networks, IPFS, Python, and Solidity—are open-source and free to use, the overall implementation cost remains low.

Infrastructure requirements are minimal, especially in the development phase, and can be scaled incrementally. The project can be deployed on public or consortium blockchains depending on budget and requirements. Moreover, the long-term savings from reduced KYC processing time and cost justify the initial investment. The solution is ideal for startups and enterprises aiming for secure, efficient KYC workflows with minimal ongoing cost. This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system is well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

3.SYSTEM ANALYSIS

3.1 specifications

The system is designed with the following high-level specifications in mind:

- User Module: Allows customers to register, log in, upload KYC documents, and monitor access.
- Admin Module: Enables bank administrators to review, approve, or reject KYC submissions and access stored records.

- **Smart Contract Module:** Handles permissions, stores KYC data hash, and logs access events.
- **Blockchain Integration:** Ensures immutability and transparency in record storage.
- **Notification Module:** Sends email alerts to users when their data is accessed.

These specifications are designed to provide a balance between functionality, security, and performance, ensuring a robust and scalable system.

3.2 Software Requirements

The software stack includes:

- **Frontend:** HTML5, CSS3, JavaScript
- **Backend:** Python (Flask/Django)
- **Blockchain Network:** Ethereum (using Ganache for local development)
- **Smart Contracts:** Solidity
- **Database:** IPFS for document storage
- **Libraries:** Web3.py or Web3.js for blockchain communication
- **Testing Frameworks:** Truffle Suite, Mocha, and PyTest
- **IDE:** Visual Studio Code, Remix IDE

All the chosen tools are open-source and widely adopted, ensuring accessibility, scalability, and developer support.

3.3 Hardware Requirements

Minimum hardware requirements for development and testing:

- **Processor:** Intel i5 or equivalent (Quad-core preferred)
- **RAM:** 8 GB or higher
- **Storage:** 256 GB SSD (for better performance)
- **Operating System:** Windows 10 / Ubuntu 20.04 or later
- **Network:** Stable internet connection for blockchain sync and API interactions

For production, cloud services like AWS or Azure may be used to host the backend with blockchain nodes and storage integrations.

3.4 Module Description

User Module:

- Register and authenticate users.
- Upload KYC documents and personal data.
- Monitor KYC status and receive notifications upon data access.

Admin (Bank) Module:

- Log in to review KYC submissions.
- Approve or reject user requests.
- Access verified user data with full transparency.

Smart Contract Module:

- Manage digital identities and KYC entries.
- Authorize or restrict access using pre-defined functions.
- Log each access in an immutable ledger.

Notification Module:

- Trigger email alerts whenever KYC data is accessed.
- Maintain user trust and transparency through communication.

Each module plays a vital role in the overall ecosystem, ensuring the secure, efficient, and decentralized management of KYC information.

4. DESIGN

4.1 Introduction

System design is a critical phase where the blueprint of the entire project is laid out. It bridges the gap between user requirements and system implementation. A well-thought-out design ensures the system is scalable, secure, modular, and aligned with business goals. This

KYC system is particularly sensitive to design quality due to the involvement of private user data, encryption, and smart contract-based workflows.

In this phase, we focus on designing components that satisfy:

- Data confidentiality and access control using cryptography
- System performance under concurrent usage
- High reliability and fault tolerance
- Compatibility with decentralized frameworks (Ethereum, IPFS)

4.2 Block Diagram

The block diagram for the "Secure Sharing of Identity (KYC)" system represents the logical layout of major system components and their interactions.

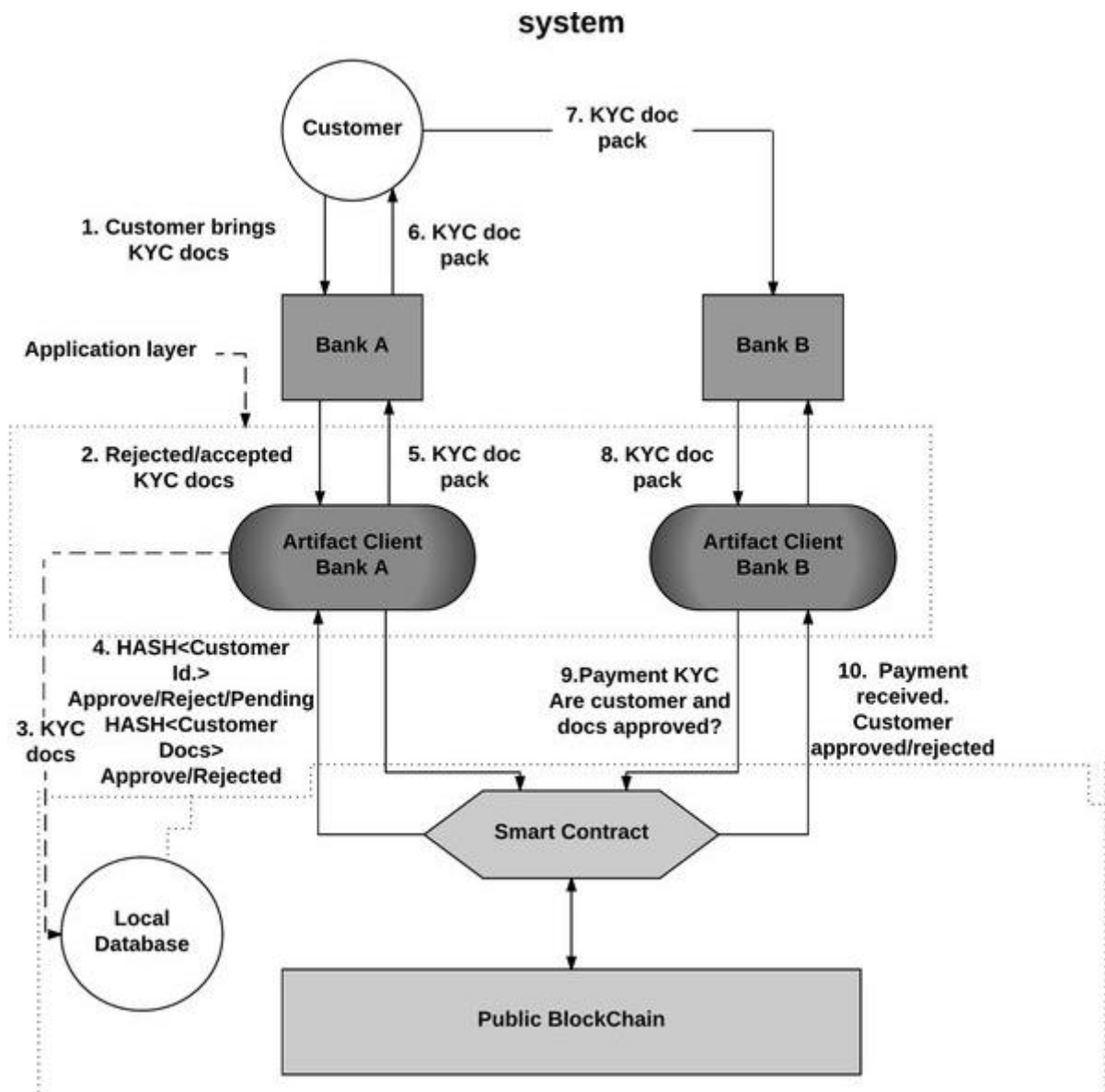
Main Components:

1. User Interface (Web App): Allows interaction with the system (upload/view KYC, track status).
2. Backend Server (Python Flask/Django): Handles API requests, authentication, and communication with the blockchain.
3. Smart Contracts: Manage permissions and automate access validation.
4. Ethereum Blockchain: Stores transactional hashes and contract states securely.
5. IPFS: Distributed document storage that prevents tampering.
6. Email Notification Service: Sends alerts to users upon access.

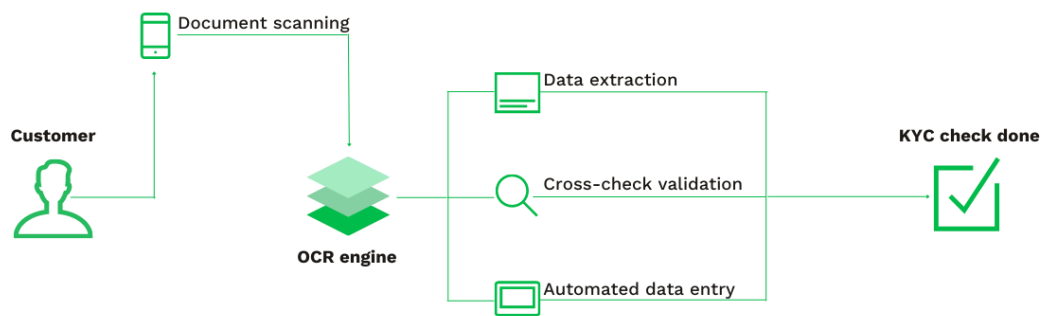
Data Flow:

- Users input data → Backend → IPFS (document) + Ethereum (hash)
- Admins request access → Smart Contract validates → Notification sent → Data fetched

This architecture ensures separation of concerns, high security, and modular expansion.



Automated KYC Process



4.3DFD / ER / UML Diagram

UML stands for Unified Modeling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modelling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems.

The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
 2. Provide extendibility and specialization mechanisms to extend the core concepts.
 3. Be independent of particular programming languages and development process.
 4. Provide a formal basis for understanding the modelling language.
 5. Encourage the growth of OO tools market.
- 7
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
 7. Integrate best practices.

Characteristics of Data Flow Diagram (DFD)

Below are some characteristics of Data Flow Diagram (DFD):

- 1. Graphical Representation:** Data Flow Diagram (DFD) use different symbols and notation to represent data flow within system. That simplify the complex system into understandable visual elements. This makes them easier to interpret by both **technical** and **non-technical stakeholders**.
- 2. Problem Analysis:** Data Flow Diagram (DFDs) are very useful in understanding a system and can be effectively used during analysis. Data Flow Diagram (DFDs) are quite general and are not limited to problem analysis for software requirements specification.
- 3. Abstraction:** DFDs abstract away the **implementation details** and focus on the **data flow and processes** within a system. They provide a high-level overview and omit unnecessary technical information.
- 4. Hierarchy:** Data Flow Diagram (DFD) provides a hierarchy of a system. High- level diagram i.e. 0-level diagram provides an overview of entire system while lower-level diagram like 1-level DFD and beyond provides a detailed data flow of individual process.

Levels in Data Flow Diagram (DFD)

DFDs are categorized into various levels, with each level providing different degrees of detail. The levels are numbered from **0** and onward. The higher the level, the more detailed the diagram becomes. The following are the four levels of DFDs:

0-Level Data Flow Diagram (DFD)

Level 0 DFD is the **highest-level diagram**, representing the system as a **single process** with its interactions with external entities. It shows the major processes, data flows, and data stores in the system, without providing any details about the internal workings of these processes. It is also known as the **Context Diagram**, which abstracts the system's operations and shows how data enters and leaves the system.

1-Level Data Flow Diagram (DFD)

Level 1 DFD provides a more detailed view of the system by breaking down the major processes identified in the level 0 Data Flow Diagram (DFD) into sub-processes. Each sub-

process is depicted as a separate process on the level 1 Data Flow Diagram (DFD). The data flows and data stores associated with each sub-process are also shown.

Level 1 DFD provides a more **detailed view** of the system, focusing on key functional aspects. The **Context Diagram** from **Level 0** is expanded into **multiple bubbles/processes**.

2-Level Data Flow Diagram (DFD)

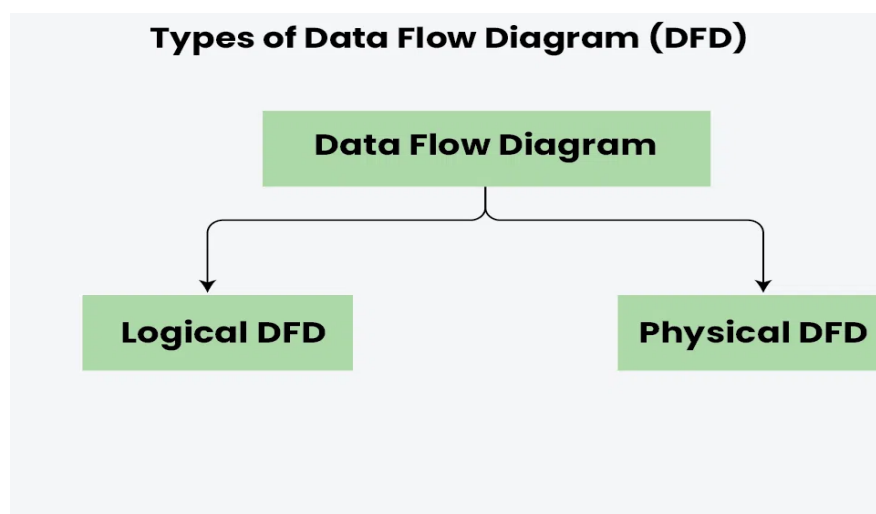
Level 2 DFD further breaks down the sub-processes from **Level 1 DFD** into **additional sub-processes**, providing an even more **detailed view**. This level is useful when dealing with **specific requirements** or parts of the system that need a closer examination of their processes and interactions.

3-Level Data Flow Diagram (DFD)

3-Level is the **most detailed level** of Data Flow Diagram (DFDs), which provides a detailed view of the processes, data flows, and data stores in the system. This level is typically used for complex systems, where a high level of detail is required to understand the system. It includes **detailed descriptions** of each process, data flow, and data store, and is usually used when there is a need for a comprehensive understanding of the system.

Types of Data Flow Diagram (DFD)

DFDs can be classified into two main types, each focusing on a different perspective of system design:



4.3.2 Context Level Diagram

Context Diagrams are high-level visual representations that show the interactions between a system being developed and its external entities, such as users, other systems, or processes. They provide a big-picture view of how the system fits into its environment without diving into the internal details of the system itself.

- Typically, they consist of a central system surrounded by external entities, with arrows representing data flow or interactions between them.
- They're useful for understanding system boundaries and dependencies.

Importance of Context Diagrams in Systems Analysis

- **Scope Definition:** Context diagrams define the system's boundaries by highlighting its interactions with external entities, ensuring that the analysis focuses on pertinent components and processes.
- **Requirement Gathering:** These diagrams visualize how the system interacts with its environment, aiding in identifying both functional and non-functional requirements. They offer clarity on the system's objectives and its external interactions.
- **Communication:** Acting as a bridge between stakeholders, such as business users, developers, and project managers, context diagrams foster shared understanding of the system's scope and context. They streamline discussions and decision-making throughout the development process.
- **Risk Identification:** Context diagrams assist in spotting potential risks stemming from the system's interactions with external entities. They help stakeholders assess the implications of external factors on the system's performance, security, and reliability.

Components of Context Diagrams

1. System/Product:

- This is the primary focus of the diagram, representing the system being analysed or designed.



Components of Context diagrams

2. External Entities

These are entities outside the system boundary that interact with the system. They could be users, other systems, or processes that provide input to or receive output from the system.



Components of Context diagrams

3. Data Flow

These are arrows representing the flow of data or information between the system and external entities. They illustrate the exchange of data between the system and its environment.

1. Unidirectional Data Flow

Unidirectional data flow indicates that data moves in only one direction between two components. For example, data flows from an external entity to a process, or

from a process to a data store, but not in the opposite direction. This type of data flow is represented by a single arrow indicating the direction of data movement.



2. Bidirectional Data Flow

Bidirectional data flow indicates that data can move in both directions between two components. For example, data flows between a process and an external entity can be bidirectional, allowing the process to send data to the external entity and receive data back. This type of data flow is represented by a double-headed arrow indicating that data can move in both directions.



4.3.3 Zero level diagrams

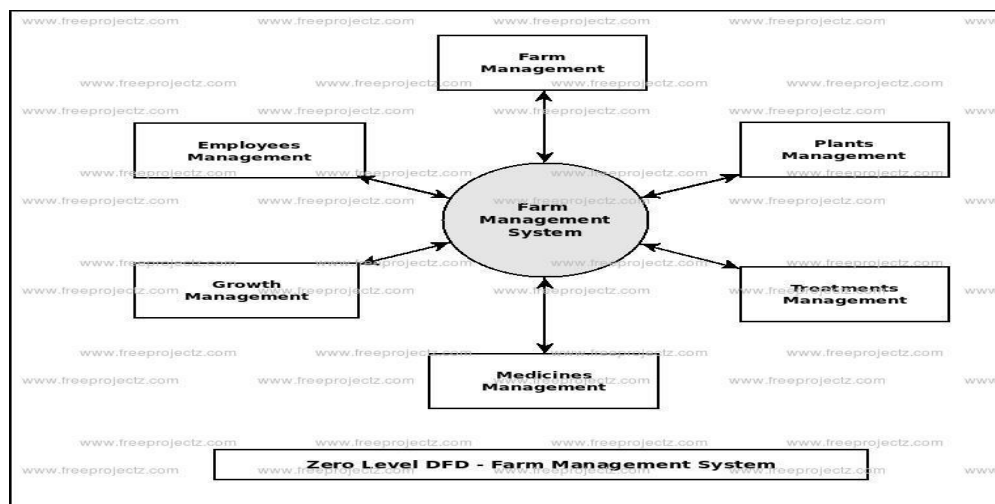
A zero-level diagram, also known as a context diagram, is the highest-level representation of a system in a Data Flow Diagram (DFD). It provides a broad overview of the system, showing its interactions with external entities without detailing internal processes.

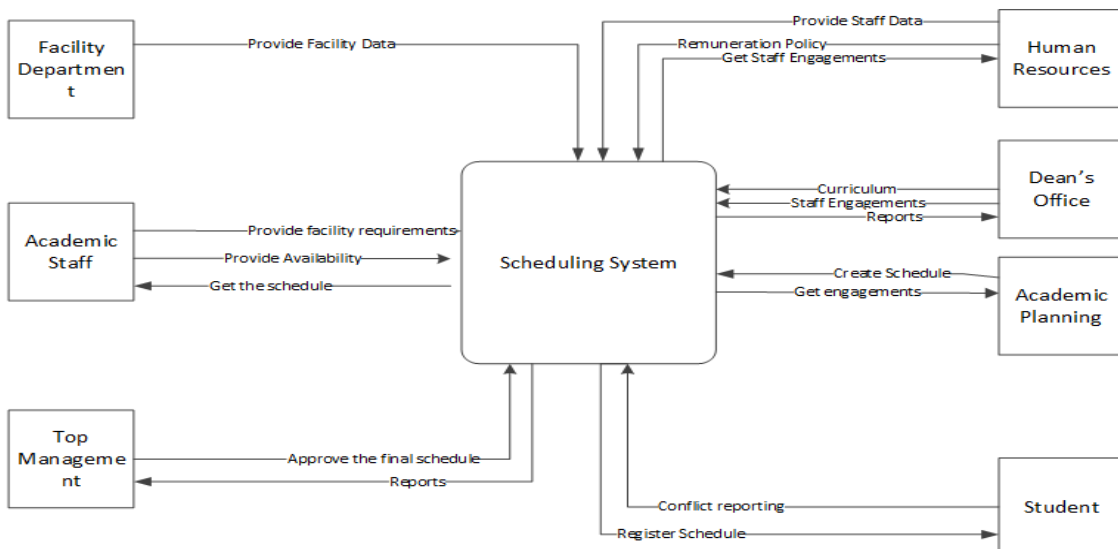
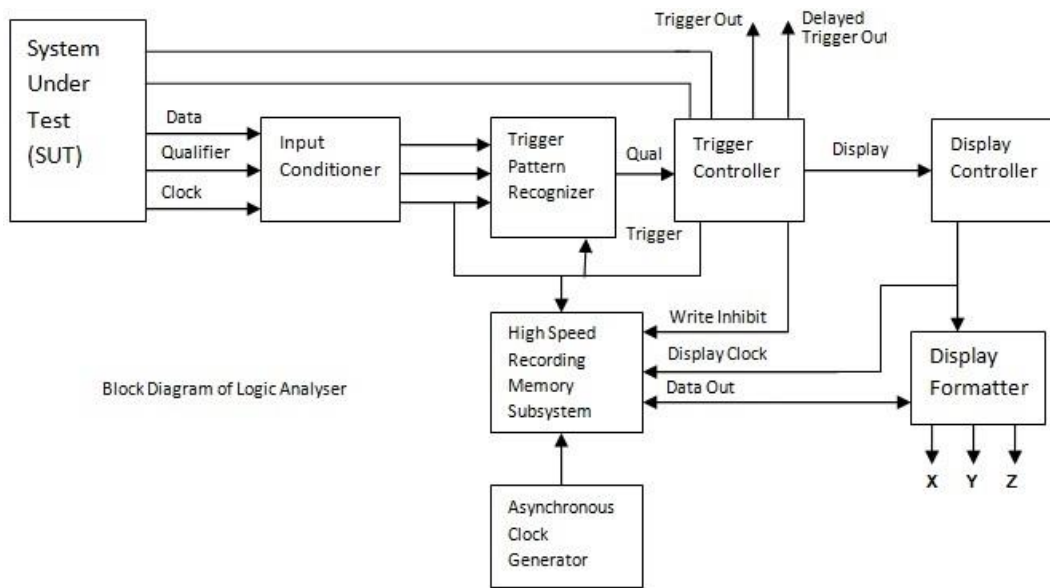
Key Features of Zero-Level Diagrams:

- **Single Process Representation:** The entire system is depicted as a single process (often represented as a circle or rectangle).
- **External Entities:** Shows interactions with external entities such as users, databases, or third-party services.
- **Data Flow:** Illustrates the flow of data between the system and external entities using arrows.
- **No Internal Details:** Does not break down the system into sub-processes—this happens in Level 1 DFDs.

Example Applications:

- **Banking Systems:** A zero-level diagram for a banking system might show interactions between customers, ATMs, and the central database.
- **KYC (Know Your Customer) Process:** In a KYC project, the diagram would depict how customer data flows between banks, regulatory bodies, and verification systems.
- **E-commerce Platforms:** It could illustrate interactions between customers, payment gateways, and inventory management.





4.4 Unified Modeling Language

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. UML is not a programming language; it is rather a visual language. Unified Modeling Language (UML) is a standardized visual modeling language that is a versatile, flexible, and user-friendly method for visualizing a system's design. Software system artifacts can be specified, visualized, built, and documented with the use of UML.

- We use UML diagrams to show the behaviour and structure of a system.
- UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.

The International Organization for Standardization (ISO) published UML as an approved standard in 2005. UML has been revised over the years and is reviewed periodically.

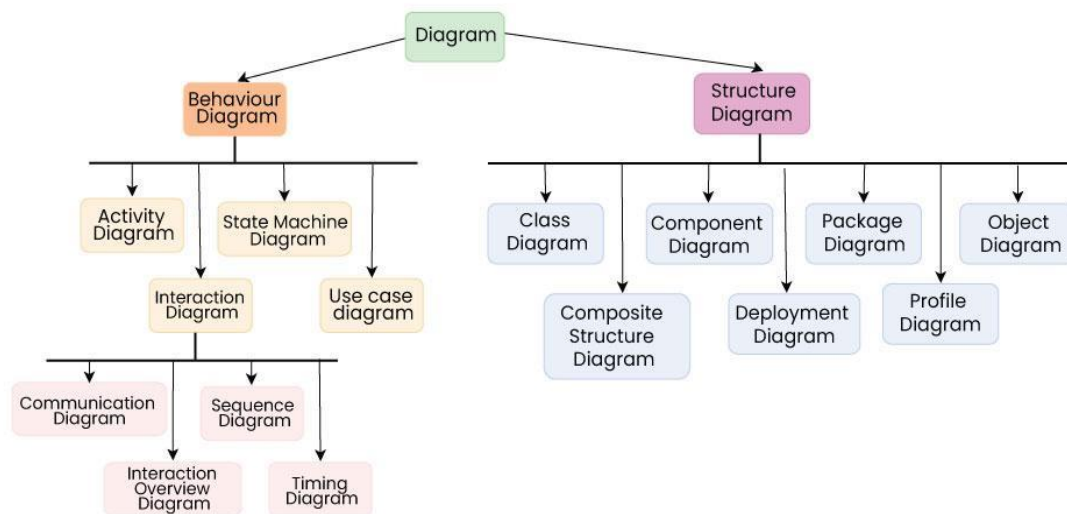
We need UML (Unified Modeling Language) to visually represent and communicate complex system designs, facilitating better understanding and collaboration among stakeholders.

Below is why we need UML:

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So, UML becomes essential to communicate with non-programmers about essential requirements, functionalities, and processes of the system.
- A lot of time is saved down the line when teams can visualize processes, user interactions, and the static structure of the system.

Types of UML Diagrams

UML is linked with object-oriented design and analysis. UML makes use of elements and forms associations between them to form diagrams. Diagrams in UML can be broadly classified as:

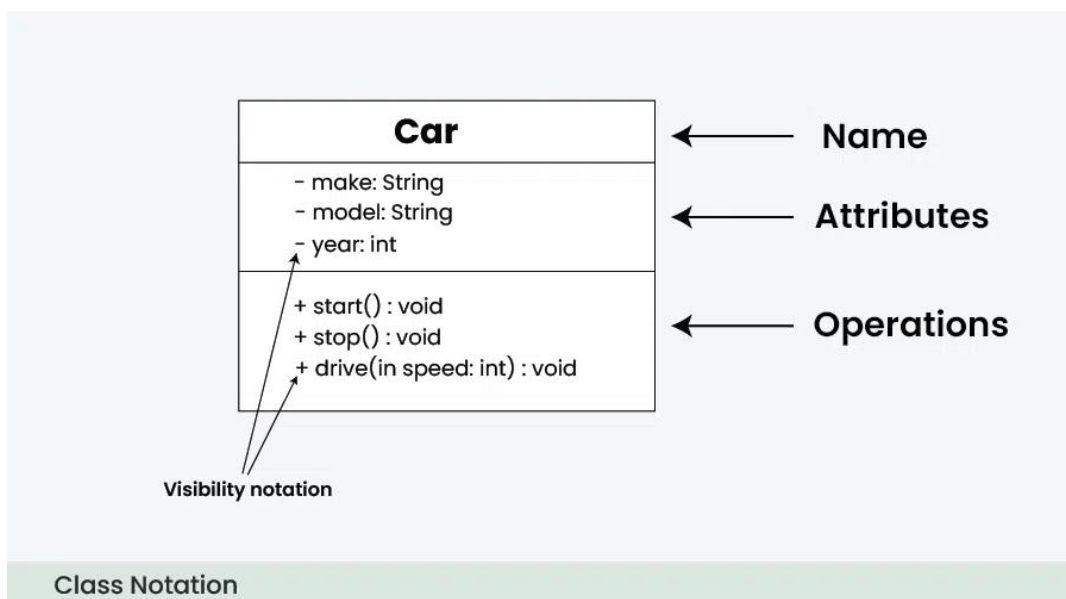
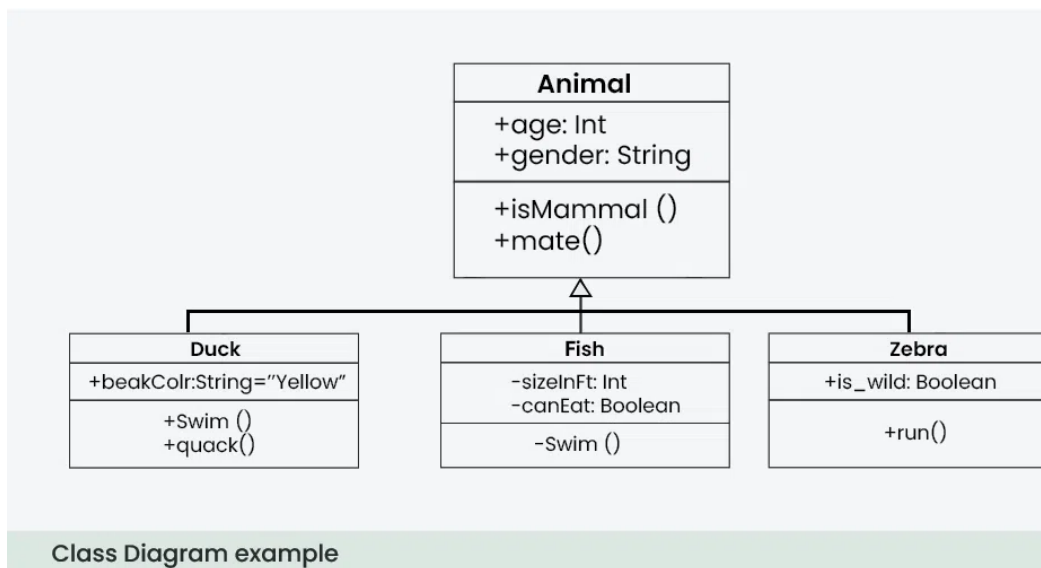


4.4.1 Class Diagram

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information. A UML class diagram is a visual tool that represents the structure of a system by showing its classes, attributes, methods, and the relationships between them. It helps everyone involved in a project—like developers and designers—understand how the system is organized and how its components interact. Class diagrams are a type of UML (Unified Modeling Language) diagram used in software engineering to visually represent the structure and relationships of classes within a system i.e. used to construct and visualize object-oriented systems.

In these diagrams, classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods. Lines connecting classes illustrate associations, showing relationships such as one-to-one or one-to-many.

Class diagrams provide a high-level overview of a system's design, helping to communicate and document the structure of the software. They are a fundamental tool in object-oriented design and play a crucial role in the software development lifecycle.



1. Class Name:

- The name of the class is typically written in the top compartment of the class box and is centered and bold.

2. Attributes:

- Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

3. **Methods:**

- Methods, also known as functions or operations, represent the behaviour or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

4. **Visibility Notation:**

- Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
 - + for public (visible to all classes)
 - - for private (visible only within the class)
 - # for protected (visible to subclasses)
 - ~ for package or default visibility (visible to classes in the same package)

4.4.2 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases.

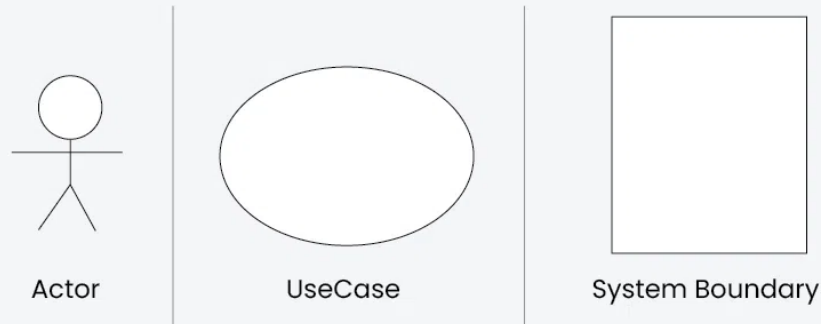
The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

A Use Case Diagram in Unified Modeling Language (UML) is a visual representation that illustrates the interactions between users (actors) and a system. It captures the functional requirements of a system, showing how different users engage with various use cases, or specific functionalities, within the system.

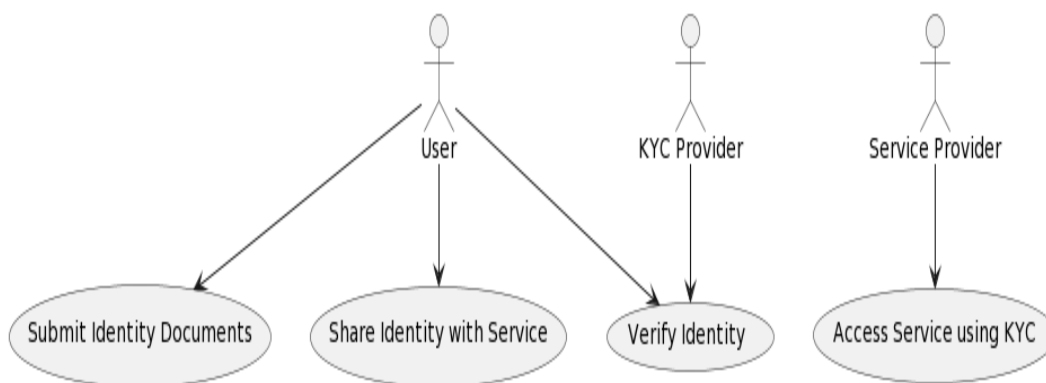
Use case diagrams provide a high-level overview of a system's behaviour, making them useful for stakeholders, developers, and analysts to understand how a system is intended to operate from the user's perspective, and how different processes relate to one another. They are crucial for defining system scope and requirements.

A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.

Use Case Diagram Notations



Use Case Diagrams | Unified Modeling Language (UML)



The Use Case Diagram offers numerous benefits throughout the system development process. Here are some key advantages of using Use Case Diagrams:

- Use Case Diagrams offer a clear visual representation of a system's functions and its interactions with external users. This representation helps stakeholders, including those without technical expertise, in grasping the system's overall behavior.
- They establish a shared language for articulating system requirements, ensuring that all team members have a common understanding.
- Use Case Diagram illustrate the different ways users engage with the system, contributing to a thorough comprehension of its functionalities.

- In the design phase, Use Case Diagrams help outline how users (actors) will interact with the system. They support the planning of user interfaces and aid in structuring system functionalities.

4.4.3 Sequence Diagram

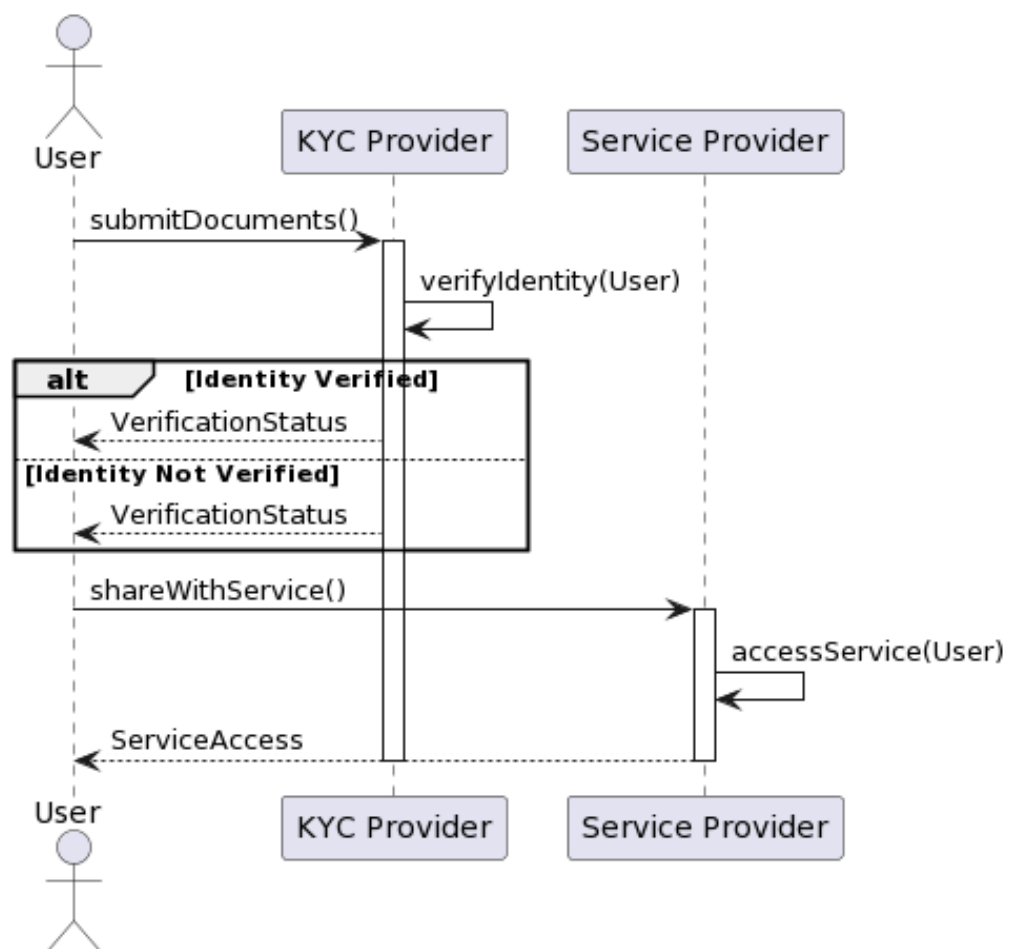
A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

Sequence diagrams are a type of UML (Unified Modeling Language) diagram that visually represent the interactions between objects or components in a system over time. They focus on the order and timing of messages or events exchanged between different system elements. The diagram captures how objects communicate with each other through a series of messages, providing a clear view of the sequence of operations or processes.

Sequence diagrams are used because they offer a clear and detailed visualization of the interactions between objects or components in a system, focusing on the order and timing of these interactions. Here are some key reasons for using sequence diagrams:

- **Visualizing Dynamic Behavior:** Sequence diagrams depict how objects or systems interact with each other in a sequential manner, making it easier to understand dynamic processes and workflows.
- **Clear Communication:** They provide an intuitive way to convey system behavior, helping teams understand complex interactions without diving into code.
- **Use Case Analysis:** Sequence diagrams are useful for analyzing and representing use cases, making it clear how specific processes are executed within a system.
- **Designing System Architecture:** They assist in defining how various components or services in a system communicate, which is essential for designing complex, distributed systems or service-oriented architectures.
- **Documenting System Behavior:** Sequence diagrams provide an effective way to document how different parts of a system work together, which can be useful for both developers and maintenance teams.

- **Debugging and Troubleshooting:** By modeling the sequence of interactions, they help identify potential bottlenecks, inefficiencies, or errors in system processes.



4.4.4 Collaboration diagram

In UML (Unified Modeling Language), a Collaboration Diagram is a type of Interaction Diagram that visualizes the interactions and relationships between objects in a system. It shows how objects collaborate to achieve a specific task or behavior. Collaboration diagrams are used to model the dynamic behavior of a system and illustrate the flow of messages between objects during a particular scenario or use case.

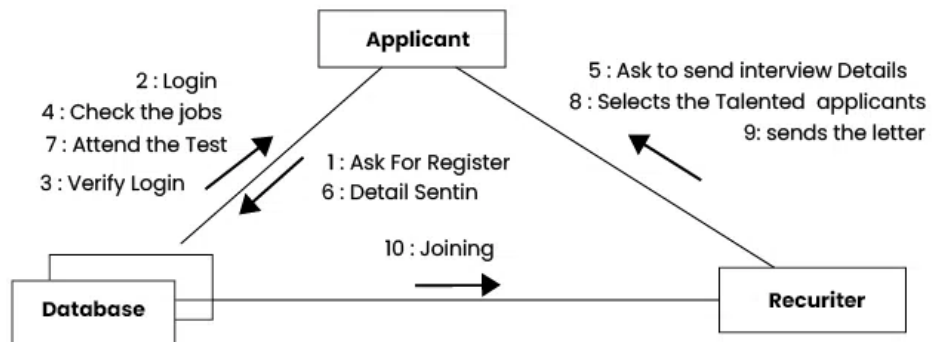
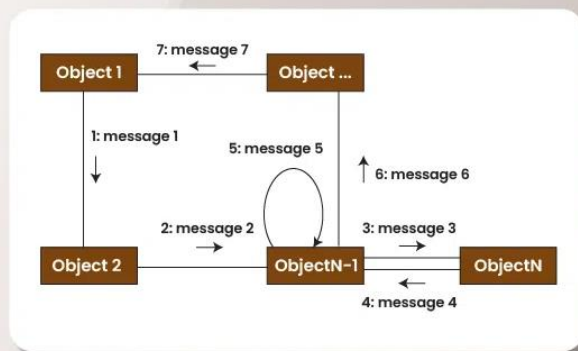
Importance of Collaboration Diagrams

Collaboration diagrams are important for understanding communication, design, analysis, and documentation of the system's architecture and behavior.

- **Visualizing Interactions:**
 - These diagrams offer a clear visual representation of how objects or components interact within a system.
 - This visualization helps stakeholders in understanding the flow of data and control for easier understanding.
- **Understanding System Behavior:**
 - By showing interactions, collaboration diagrams provide insights into the system's dynamic behavior during operation.
 - Understanding this behavior is important for identifying potential issues, optimizing performance, and ensuring the system functions smoothly.
- **Facilitating Communication:**
 - Collaboration diagrams provide an effective communication tools among team members.
 - They facilitate discussions, enabling refinement of the system's design, architecture, and functionality.

In simpler terms, they visually represent the interactions between objects or components in a system and show how they collaborate to accomplish tasks within a system, and also illustrate the system's object architecture.

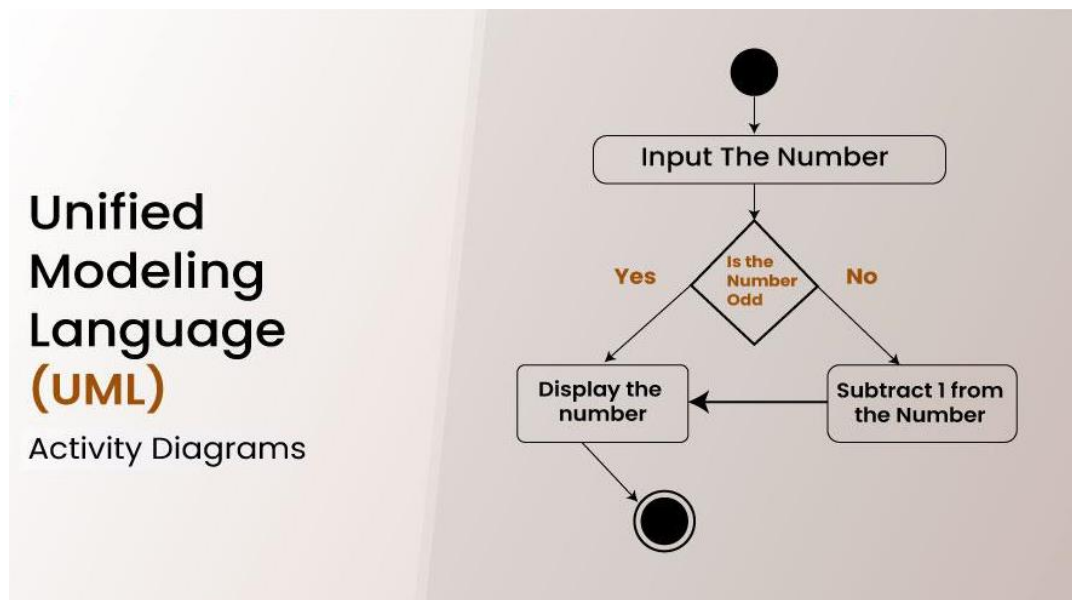
Collaboration Diagrams in Unified Modeling Language(UML)



- Keeping collaboration diagrams clear in large systems with many objects and interactions can be tough.
- Sometimes, interpreting the interactions in diagrams isn't straight forward, leading to mis-understandings.
- Diagrams might not fully capture systems where interactions change over time, like those with continuously processing and improvising.
- Ensuring that diagrams effectively convey complex interactions to all stakeholders can be challenging.

4.4.5 Activity Diagram

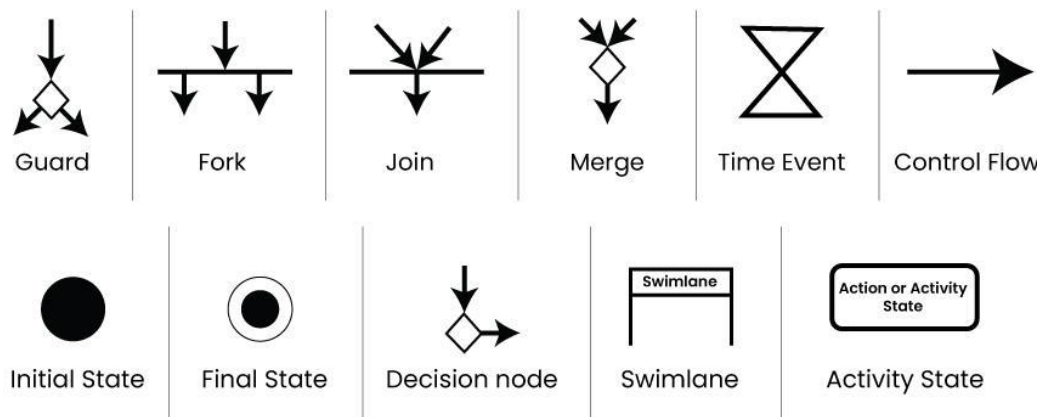
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control. Activity diagrams are an essential part of the Unified Modeling Language (UML) that help visualize workflows, processes, or activities within a system. They depict how different actions are connected and how a system moves from one state to another. By offering a clear picture of both simple and complex workflows, activity diagrams make it easier for developers and stakeholders to understand how various elements interact in a system.



Activity diagrams are useful in several scenarios, especially when you need to visually represent the flow of processes or behaviors in a system. Here are key situations when you should use an activity diagram:

1. **Modeling Workflows or Processes:** When you need to map out a business process, workflow, or the steps involved in a use case, activity diagrams help visualize the flow of activities.
2. **Concurrent or Parallel Processing:** If your system or process involves activities happening simultaneously, an activity diagram can clearly show the parallel flow of tasks.
3. **Understanding the Dynamic Behavior:** When it's essential to depict how a system changes over time and moves between different states based on events or conditions, activity diagrams are effective.

4. **Clarifying Complex Logic:** Use an activity diagram to simplify complex decision-making processes with branching paths and different outcomes.
5. **System Design and Analysis:** During the design phase of a software system, activity diagrams help developers and stakeholders understand how different parts of the system interact dynamically.
6. **Describing Use Cases:** They are useful for illustrating the flow of control within a use case, showing how various components of the system interact during its execution.

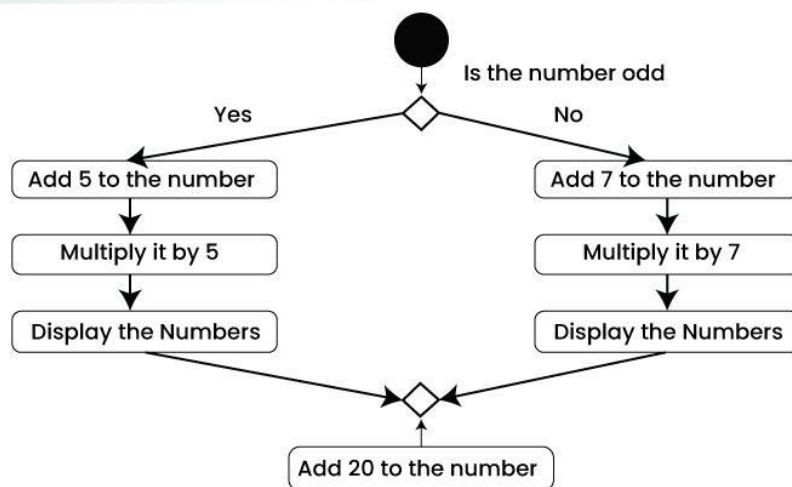


Activity Diagram Notations

For example:

In the diagram below: we can't have both sides executing concurrently, but they finally merge into one. A number can't be both odd and even at the same time.

An Activity Diagram using Merge Notation



5. IMPLEMENTATION

What is Python:

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard libraries which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia
-

Advantages of Python: -

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be **extended to other languages**. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add **scripting capabilities** to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers **more productive** than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

When working with Java, you may have to create a class to print '**Hello World**'. But in Python, just a print statement will do. It is also quite **easy to learn, understand, and code**. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and **indentation is mandatory**. These further aids the readability of the code.

8. Object-Oriented

This language supports both the **procedural and object-oriented** programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the **encapsulation of data** and functions into one.

9. Free and Open-Source

Like we said earlier, Python is **freely available**. But not only can you **download Python** for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to **code only once**, and you can run it anywhere. This is called **Write Once Run Anywhere (WORA)**. However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, **debugging is easier** than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

Advantages of Python Over Other Languages:

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 GitHub annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and **machine learning**, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in **slow execution**. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbonnelle**.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically-typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck.

While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC (Java Database Connectivity)** and **ODBC (Open Database Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java; I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

What is Machine Learning: -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning: -

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning. *Supervised learning* involves somehow

modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities.

We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as *clustering* and *dimensionality reduction*. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning: -

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason

behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality – Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning: -

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML –

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting

- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping.

How to Start Learning Machine Learning?

Arthur Samuel coined the term “**Machine Learning**” in 1959 and defined it as a “**Field of study that gives computers the capability to learn without being explicitly programmed**”.

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to Indeed, Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and an average base salary of **\$146,085** per year.

But there is still a lot of doubt about what exactly is Machine Learning and how to start learning it? So, this article deals with the Basics of Machine Learning and also the path you can follow to eventually become a full-fledged Machine Learning Engineer. Now let's get started!!!

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisite: case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So, it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like colour, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

(b) Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labelled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabelled data like Unsupervised Learning with a small amount of labelled data. Using labelled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.
- **Reinforcement Learning** – This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning: -

1. Easily identifies trends and patterns -

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As **ML algorithms** gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning: -

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfil their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

Purpose: -

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy

that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules Used in Project: -

TensorFlow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy: NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions

- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. **Python**

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac:

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac:

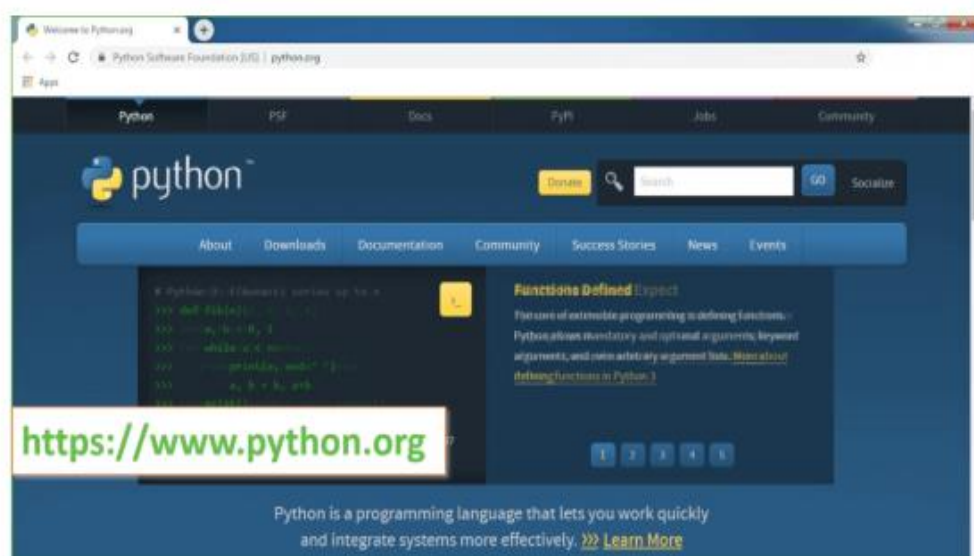
There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. [Download the Python Cheat sheet here.](#) The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	 Download	Release Notes
Python 3.6.9	July 2, 2019	 Download	Release Notes
Python 3.7.3	March 25, 2019	 Download	Release Notes
Python 3.4.10	March 18, 2019	 Download	Release Notes
Python 3.5.7	March 18, 2019	 Download	Release Notes
Python 2.7.16	March 4, 2019	 Download	Release Notes
Python 3.7.2	Dec. 24, 2018	 Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files					
Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		68111671e5b2db4ae7b9ab01bf09be	23017663	SG
XZ compressed source tarball	Source release		d33e4aaf6097051c2eca45ee3604803	17131432	SG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fa7583daf1a442cbalcee08e6	34898416	SG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a45773bf5e4a936b241f	28082845	SG
Windows help file	Windows		d63999573a2c56b2ac56cade6b477cfd2	8131761	SG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	980b3cfd9ec0bfabe63184a40728a2	7504391	SG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b4b0ad76debdb3043a183e563400	26488368	SG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28cb1c608b6d73aebef3a3bd351b4bd2	1362904	SG
Windows x86 embeddable zip file	Windows		9fab3bd18b41879fda94113574139d0	6741826	SG
Windows x86 executable installer	Windows		33cc802942a5444a3d9451478394789	25663848	SG
Windows x86 web-based installer	Windows		1b670cfa5d317df82c30983ea371d87c	1324608	SG

- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

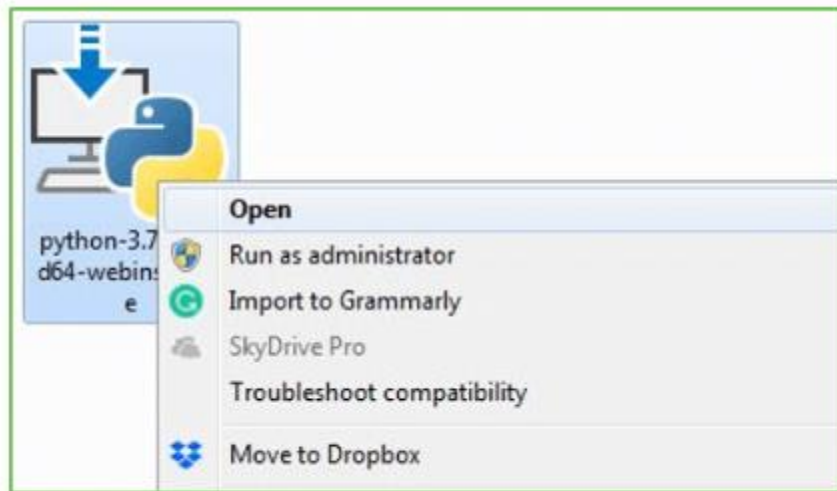
- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, make sure to put a tick on Add Python 3.7 to PATH



Step 3: Click on Install NOW After the installation is successful. Click on Close.



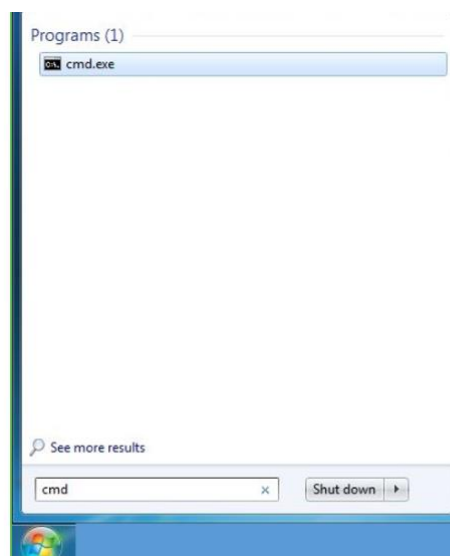
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

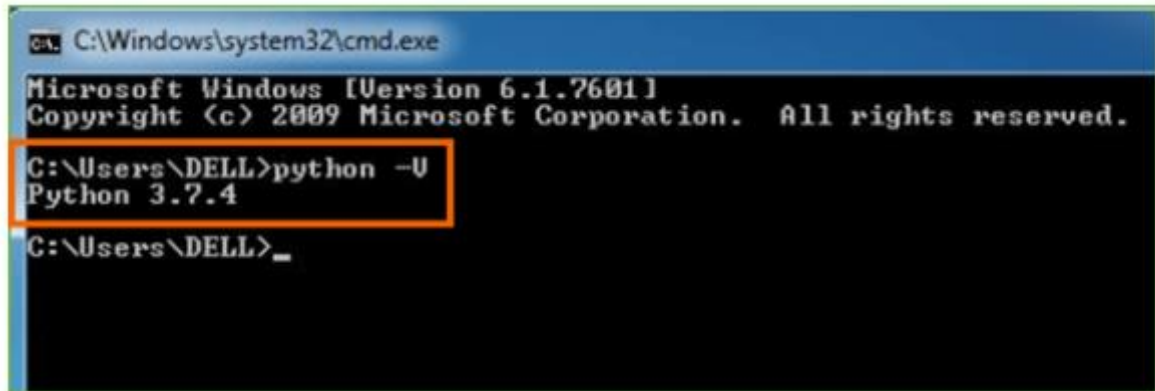
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python -V** and press Enter.



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\DELL>python -U
Python 3.7.4

C:\Users\DELL>_
```

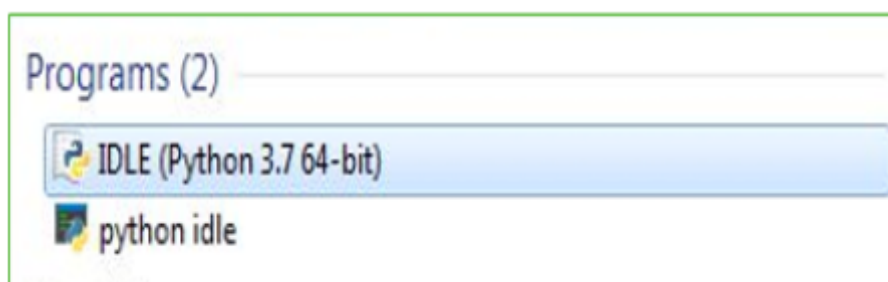
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

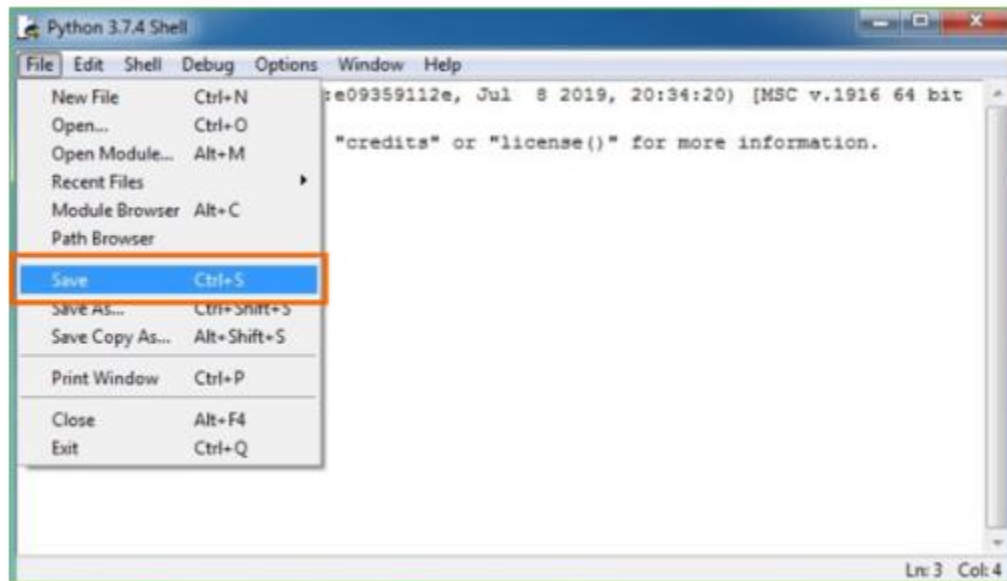
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print

6.TESTING

6.1 Introduction

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests

perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of

system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

6.2 Design of Test Cases and Scenarios

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed

- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

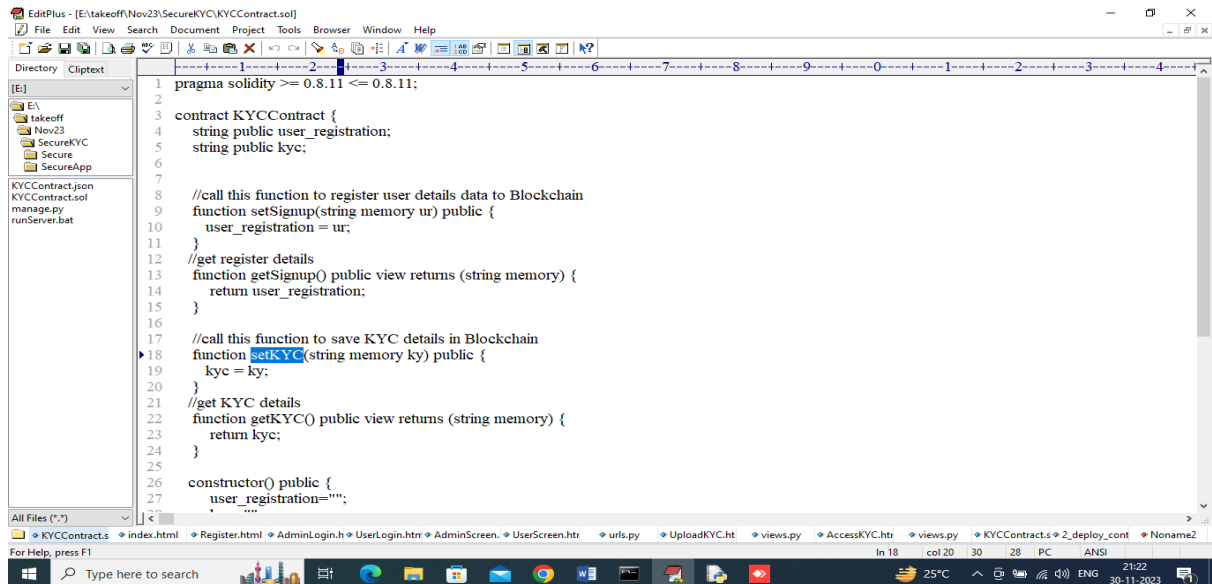
Now-a-days worldwide 70% user's data are available online and this data will be stored at single centralized servers whose owners or database managers may misuse or alter user's data and there is no direct way for the user to know about such misuse. Sometime for Identity proof or KYC registrations users may have to disclose sensitive information to such servers and leakage of such sensitive information may harm user's privacy. Sensitive information include address, zip code, gender, diseases etc.

To avoid such leakage of sensitive information we are employing Blockchain based KYC Sharing. Blockchain has inbuilt support for data encryptions, data access control and immutability (data once stored can be altered in any manner) of stored data. Blockchain store each record as transaction or block and associate each block with unique HASHCODE and while storing fresh records Blockchain will verify HASHCODE of all previous records and if data unchanged then it will result into successful verification and this verification will make Blockchain immutable. Data stored in Blockchain can be access by only those users who has permissions and can be consider as Secure Sharing of Records. In propose work if any company or other organization access user KYC then Blockchain will send EMAIL notification to KYC user about access.

Above advantages of Blockchain diverting us to manage all KYC records using Blockchain. Blockchain can store or retrieve data using Smart Contract which can be designed using Solidity

code. Smart Contract contains functions which will be access by Blockchain to manage user data. In below screen we are showing smart contract code

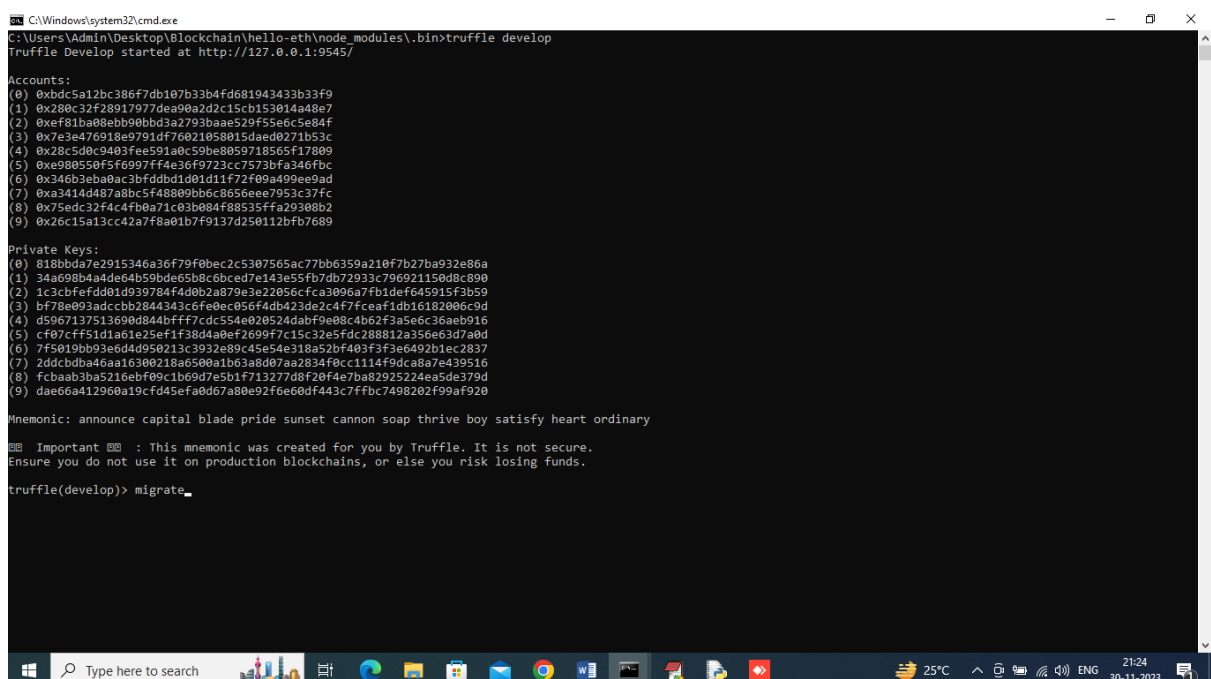
7. OUTPUT SCREENS



```
1 pragma solidity >= 0.8.11 <= 0.8.11;
2
3 contract KYCContract {
4     string public user_registration;
5     string public kyc;
6
7
8     //call this function to register user details data to Blockchain
9     function setSignup(string memory ur) public {
10         user_registration = ur;
11     }
12     //get register details
13     function getSignup() public view returns (string memory) {
14         return user_registration;
15     }
16
17     //call this function to save KYC details in Blockchain
18     function setKYC(string memory ky) public {
19         kyc = ky;
20     }
21     //get KYC details
22     function getKYC() public view returns (string memory) {
23         return kyc;
24     }
25
26     constructor() public {
27         user_registration="";
28     }
29 }
```

In above contract we have defined function to manage user sign up details and KYC details and this contract need to deploy in Blockchain using below steps

- 1) First go inside 'hello-eth/node-modules/bin' folder and then find for 'runBlockchain.bat' file and then double click on that file to get below Ethereum screen
- 2)



```
C:\Windows\system32\cmd.exe
C:\Users\Admin\Desktop\Blockchain\hello-eth\node_modules\bin>truffle develop
Truffle Develop started at http://127.0.0.1:9545/

Accounts:
(0) 0xbdc5a12bc386f7db107b33b4fd6819433b33f9
(1) 0x280e32f28917977de9862d2c15cb159014e48e7
(2) 0xf81ba08eb9b0bd3a27930baae72f55e6c8e84f
(3) 0x7e3e476918e9791df76021058015daed0271b53c
(4) 0x28c5d0c9403fee591a0c59be805971856f17809
(5) 0xe980550f5f6997ff4e36f9723cc7573bfa346fbc
(6) 0x346b3eba0ac3bfddbd1d01d11f72f09a499ee9ad
(7) 0xa3414d487a8bc5f48809bb6c8656ee7953c37fc
(8) 0x75edc32f4c4fb0a71c03b084f88535ffa29308b2
(9) 0x26c15a13cc42a7f8a01b7f9137d250112bf7689

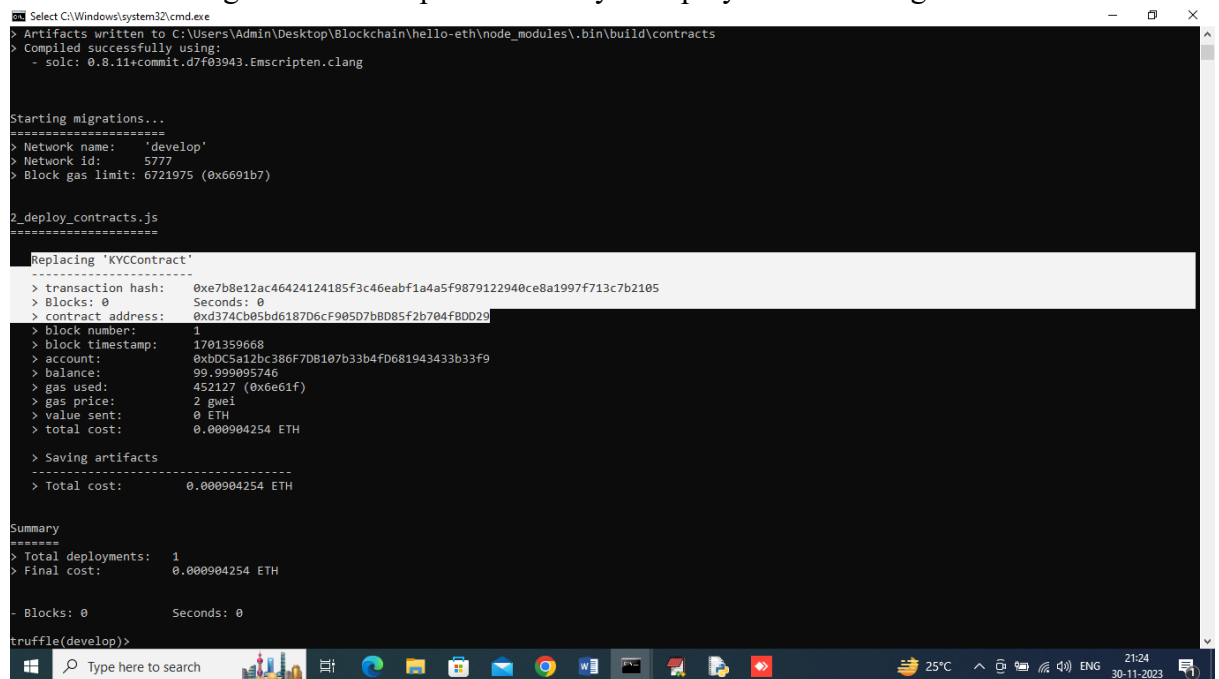
Private Keys:
(0) 818bbda7e2915346a36f79f0bec2c5307565ac77bb6359a210f7b27ba932e86a
(1) 34a698b4a4de64b59bde65b8c6bcd7e143e55fb7db72933c796921150d8c890
(2) 1c3cbfdd01d939784f4d0b2a879e3e22056cfca3096a7fb1def645915f3b59
(3) bf78e093adcbb2844343c6fe0ec056f4db423de2c4f7fcea1db16182006c9d
(4) d5967137513690d844bfff7cd554e020524dabf9e08c4b62f3a5e6c36aeb916
(5) cf07cf51d1a61e25ef1f38d4a0ef2699f7c15c32e5fcd288812a356e3d7a0d
(6) 7f5019bb93e6d4d0521c3932e09c45e54e318e52bf403f3a6e492b1ec2037
(7) 2ddcbbba46aa16300218a6500a1b63a8d07aa2834f08c1114f9dca8a7e439516
(8) fchaab3ba5216ebf09c1b69d7e5b1f713277d8f20f4e7ba82925224ea5de379d
(9) dae66a412960a19cf4d5efa0d67a80e92f6e60df443c7ffbc749820f99af920

Mnemonic: announce capital blade pride sunset cannon soap thrive boy satisfy heart ordinary

Important : This mnemonic was created for you by Truffle. It is not secure.
Ensure you do not use it on production blockchains, or else you risk losing funds.

truffle(develop)> migrate_
```

- 3) In above screen Ethereum started with default accounts and private keys and then type command as 'migrate' and then press enter key to deploy contract and get below screen



```
Select C:\Windows\system32\cmd.exe
> Artifacts written to C:\Users\Admin\Desktop\Blockchain\Hello-eth\node_modules\.bin\build\contracts
> Compiled successfully using:
  - solc: 0.8.11+commit.d7f89943.Emscripten.clang

Starting migrations...
=====
> Network name:    'develop'
> Network id:     5777
> Block gas limit: 6721975 (0x6691b7)

2_deploy_contracts.js
=====
> Replacing 'KYCContract'
> transaction hash: 0xe7b8e12ac46424124185f3c46eabf1a4a5f9879122940ce8a1997f713c7b2105
> Blocks: 0
> contract address: 0xd374Cb05bd6187D6cF905D7bBD85f2b704fBD029
> block number: 1
> block timestamp: 1701359668
> account: 0xbDC5a12bc386F7D8107b33b4fD681943433b33f9
> balance: 99.999095746
> gas used: 452127 (0x6e61f)
> gas price: 2 gwei
> value sent: 0 ETH
> total cost: 0.000904254 ETH

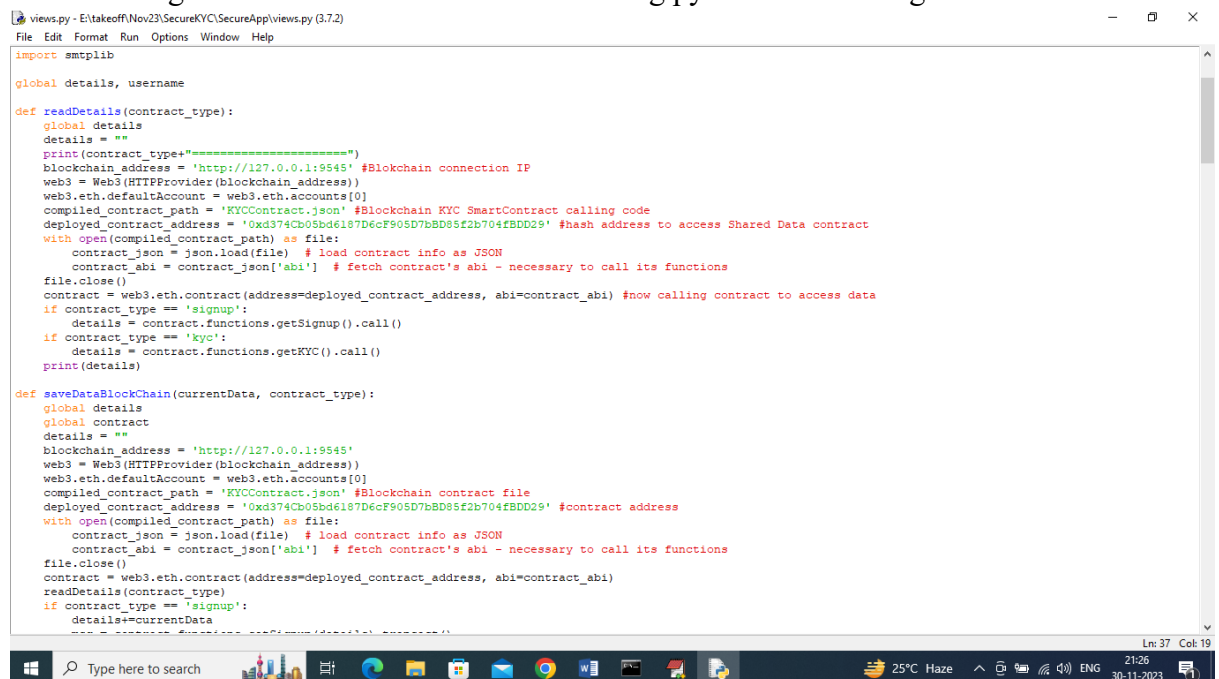
> Saving artifacts
> Total cost: 0.000904254 ETH

Summary
=====
> Total deployments: 1
> Final cost: 0.000904254 ETH

- Blocks: 0          Seconds: 0

truffle(develop)>
```

- 4) In above screen in white colour text can see KY Contract deployed and got contract address also and this address need to specify in PYTHON code to access that contract to store and get user data. In below screen showing python code calling smart contract



```
views.py - E:\takeoff\Nov23\SecureKYC\SecureApp\views.py (3.7.2)
File Edit Format Run Options Window Help

import smtplib

global details, username

def readDetails(contract_type):
    global details
    details = ""
    print(contract_type+"=====")
    blockchain_address = 'http://127.0.0.1:9545' #Blockchain connection IP
    web3 = Web3(HTTPProvider(blockchain_address))
    web3.eth.defaultAccount = web3.eth.accounts[0]
    compiled_contract_path = 'KYCContract.json' #Blockchain KYC SmartContract calling code
    deployed_contract_address = '0xd374Cb05bd6187D6cF905D7bBD85f2b704fBD029' #hash address to access Shared Data contract
    with open(compiled_contract_path) as file:
        contract_json = json.load(file) # load contract info as JSON
        contract_abi = contract_json['abi'] # fetch contract's abi - necessary to call its functions
    file.close()
    contract = web3.eth.contract(address=deployed_contract_address, abi=contract_abi) #now calling contract to access data
    if contract_type == 'signup':
        details = contract.functions.getSignup().call()
    if contract_type == 'kyc':
        details = contract.functions.getKYC().call()
    print(details)

def saveDataBlockChain(currentData, contract_type):
    global details
    global contract
    details = ""
    blockchain_address = 'http://127.0.0.1:9545'
    web3 = Web3(HTTPProvider(blockchain_address))
    web3.eth.defaultAccount = web3.eth.accounts[0]
    compiled_contract_path = 'KYCContract.json' #Blockchain contract file
    deployed_contract_address = '0xd374Cb05bd6187D6cF905D7bBD85f2b704fBD029' #contract address
    with open(compiled_contract_path) as file:
        contract_json = json.load(file) # load contract info as JSON
        contract_abi = contract_json['abi'] # fetch contract's abi - necessary to call its functions
    file.close()
    contract = web3.eth.contract(address=deployed_contract_address, abi=contract_abi)
    readDetails(contract_type)
    if contract_type == 'signup':
        details+=currentData
    else:
        details+=currentData+currentData

# def saveDataBlockChain(currentData, contract_type):
```

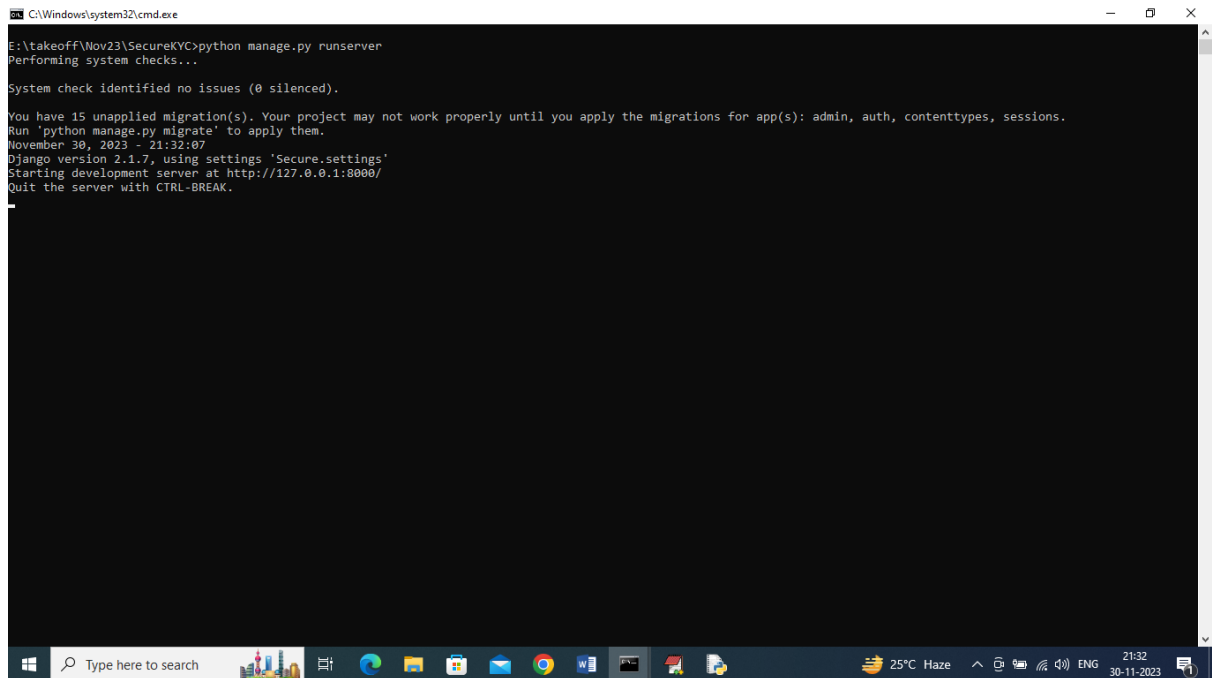
- 6) In above screen read red colour comments to know about contract calling and now contract deployed and let that contract running.

To implement this project, we have designed following modules

- 1) User module: user can sign up with the application and then login and can upload KYC details with Identity proof image and can check status of KYC and can get notification on Registered EMAIL about KYC access by which organization

- 2) **Bank Admin Module:** this user can sign up and login to application and then can view list of uploaded KYC and after all verification Bank Admin may Accept or Reject user KYC. All this KYC can be access by only those Bank Admin who registered with Blockchain and if any organization access any User KYC then Blockchain will forward mail notification.

To run project double click on 'runServer.bat' file to start python server and get below page

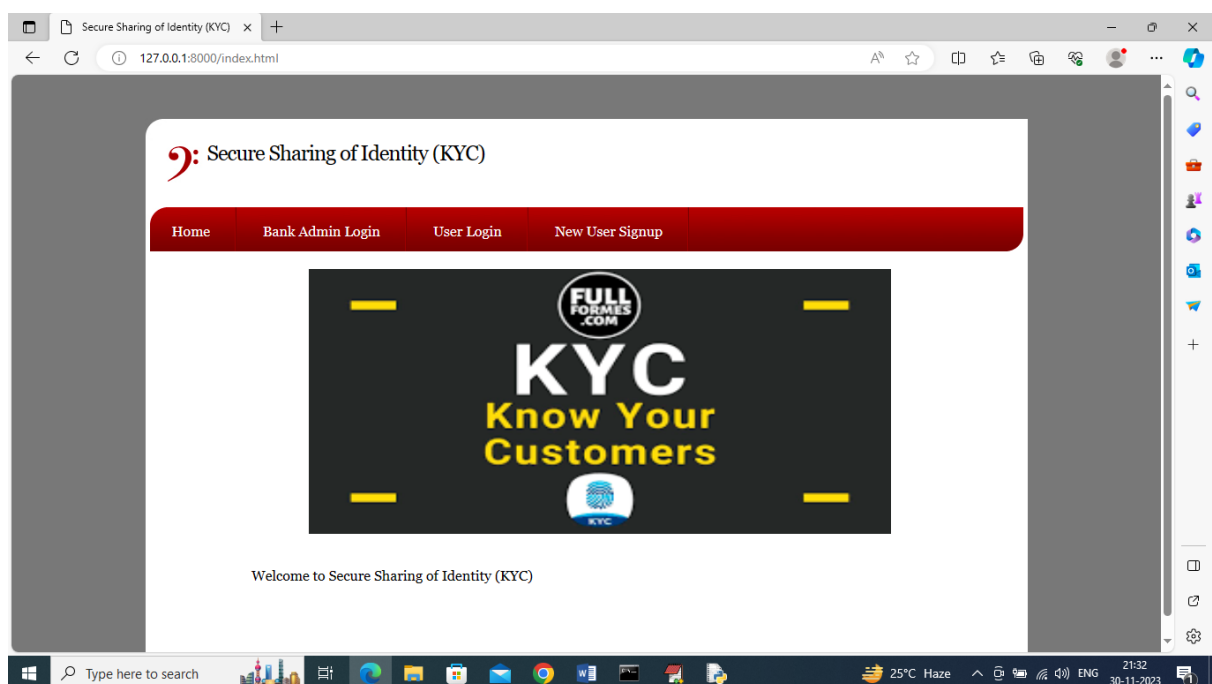


```
C:\Windows\system32\cmd.exe
E:\takeoff\Nov23\SecureKYC>python manage.py runserver
Performing system checks...

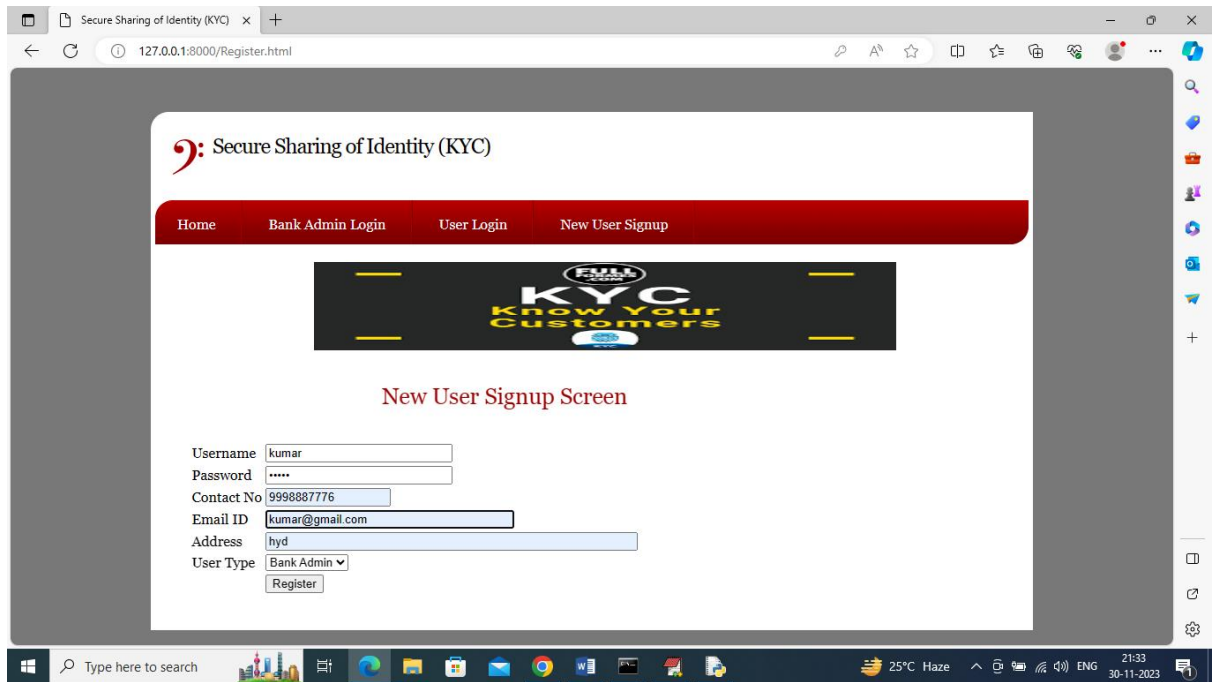
System check identified no issues (0 silenced).

You have 15 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
November 30, 2023 - 21:32:07
Django version 2.1.7, using settings 'Secure.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

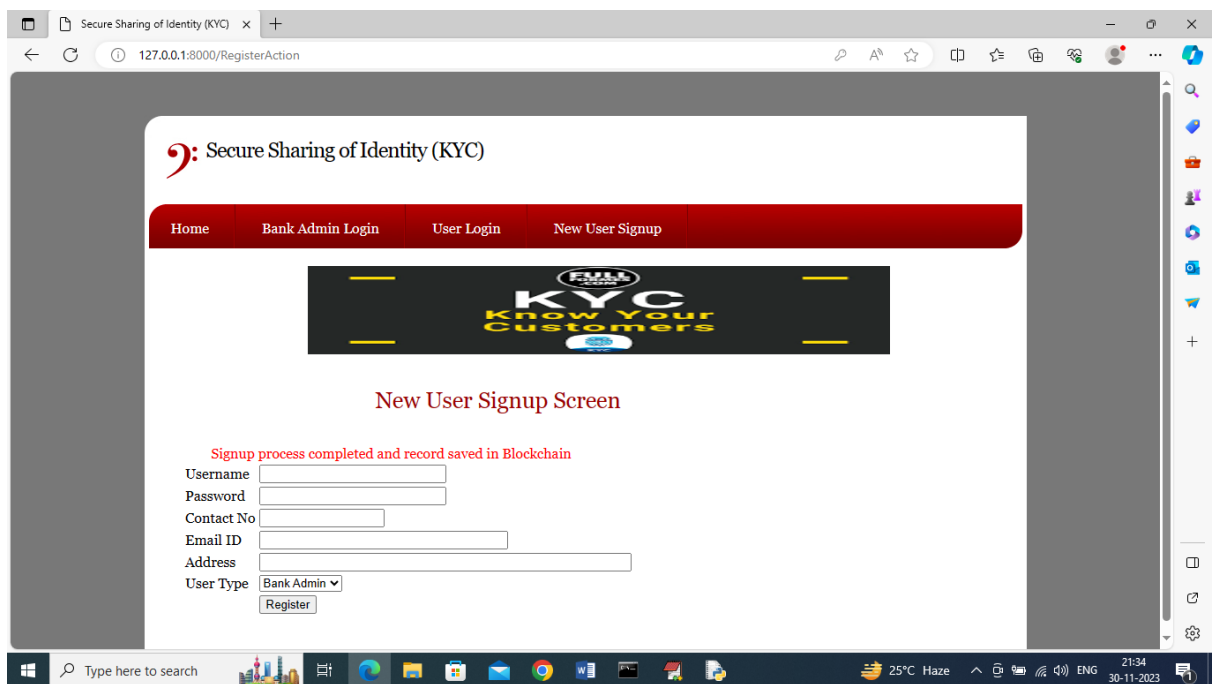
In above screen python server started and now open browser and enter URL as 'http://127.0.0.1:8000/index.html' and press enter key to get below page



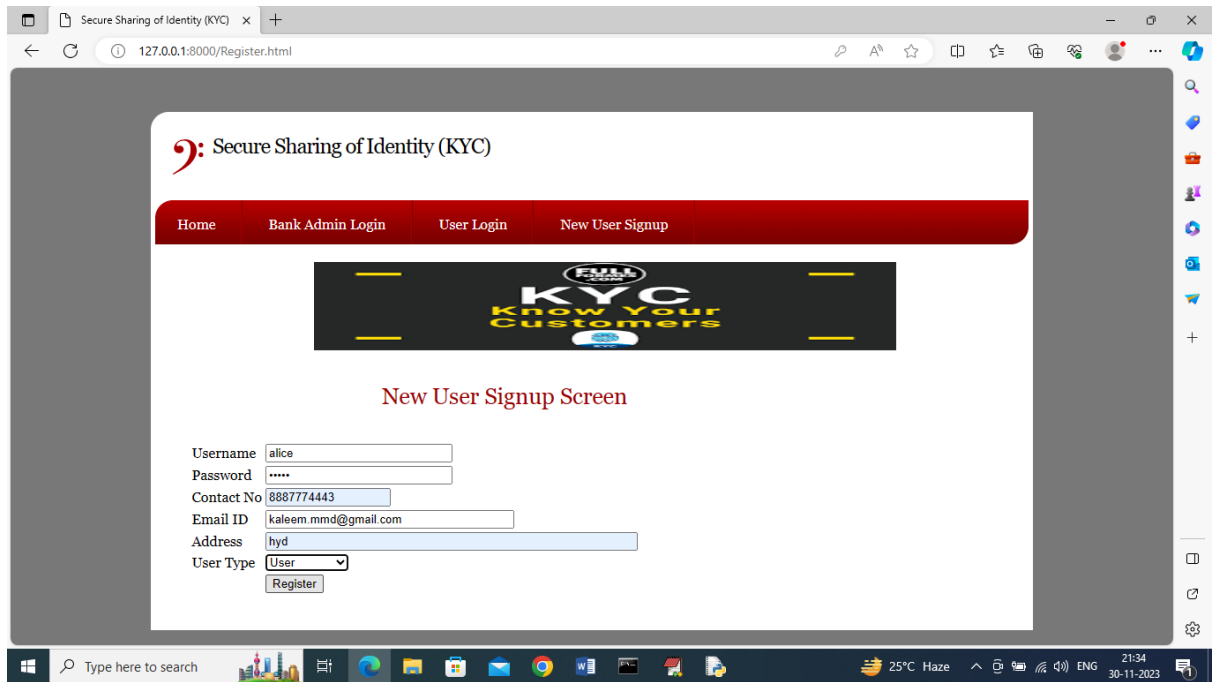
In above screen click on 'New User Signup' to add users to Blockchain and get below page



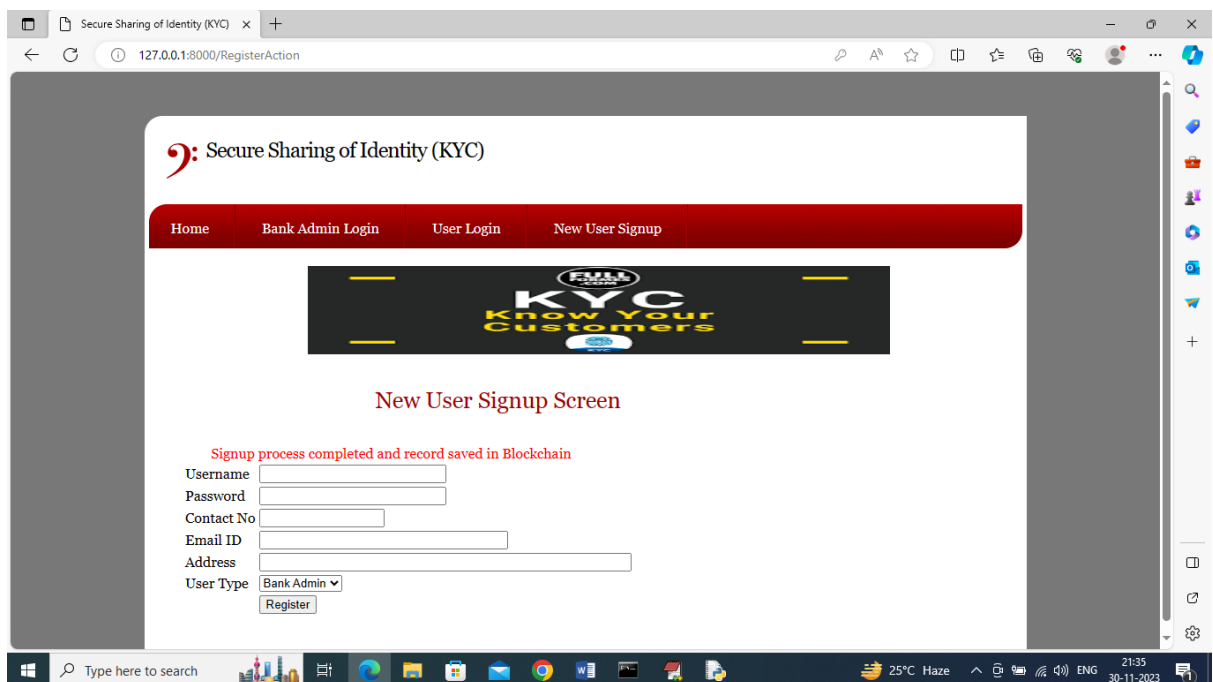
In above screen user is entering details as Bank Admin and then press button to get below page



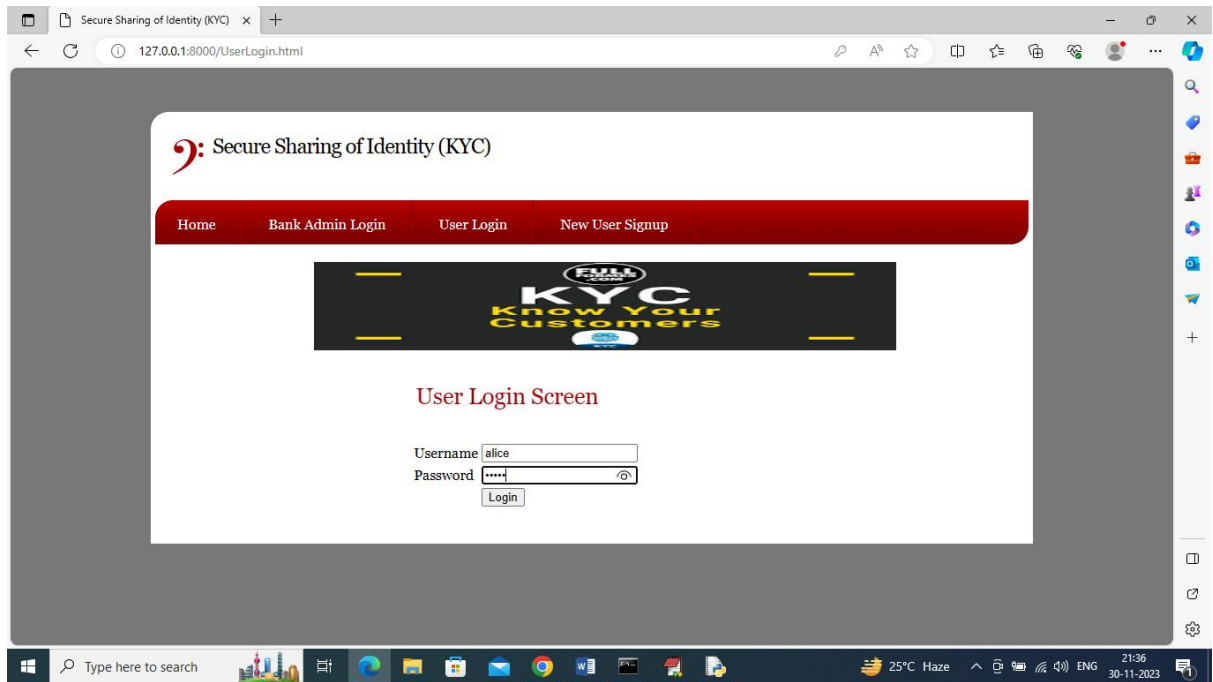
In above screen Bank Admin sign up completed and similarly add one user like below screen



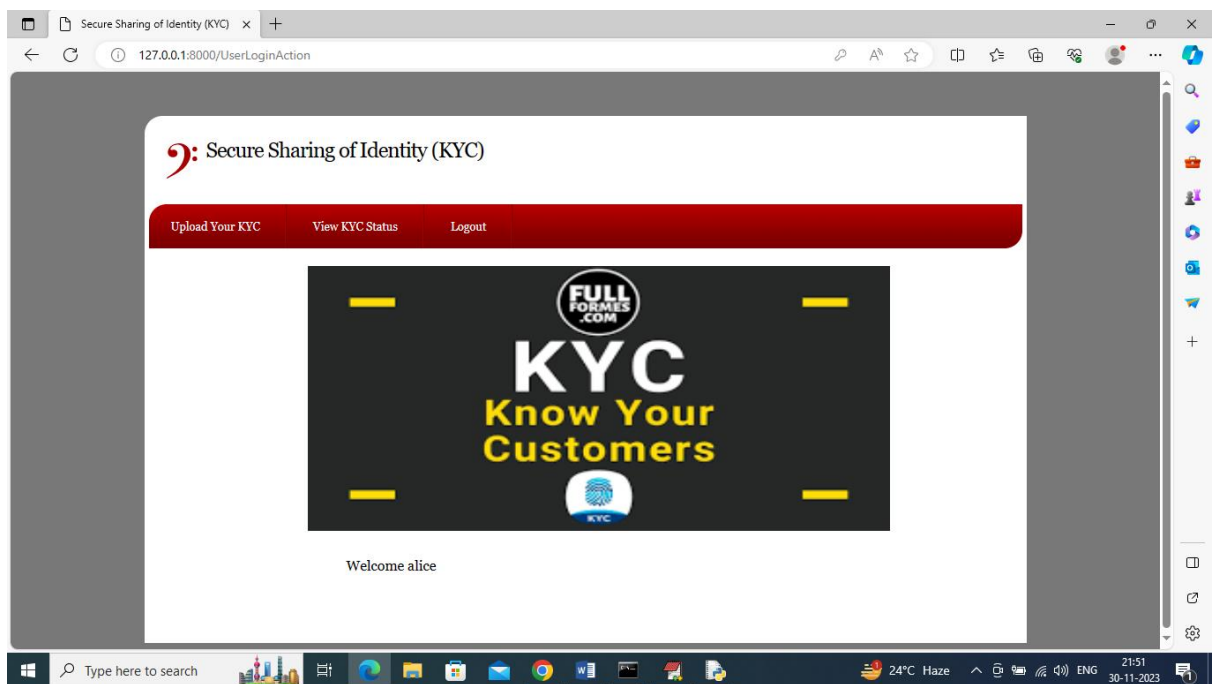
In above screen adding another details as 'User' and enter valid email ID to get notification and then press button to get below page



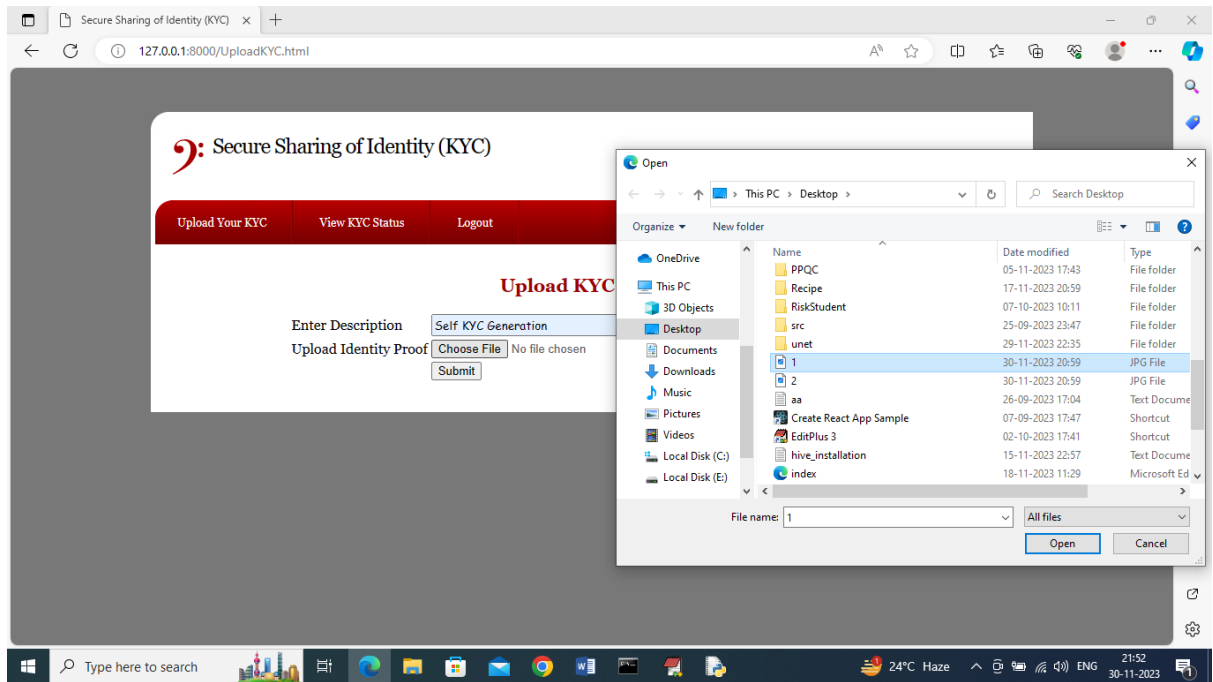
In above screen user sign up also completed and now click on 'User Login' link to get below login screen



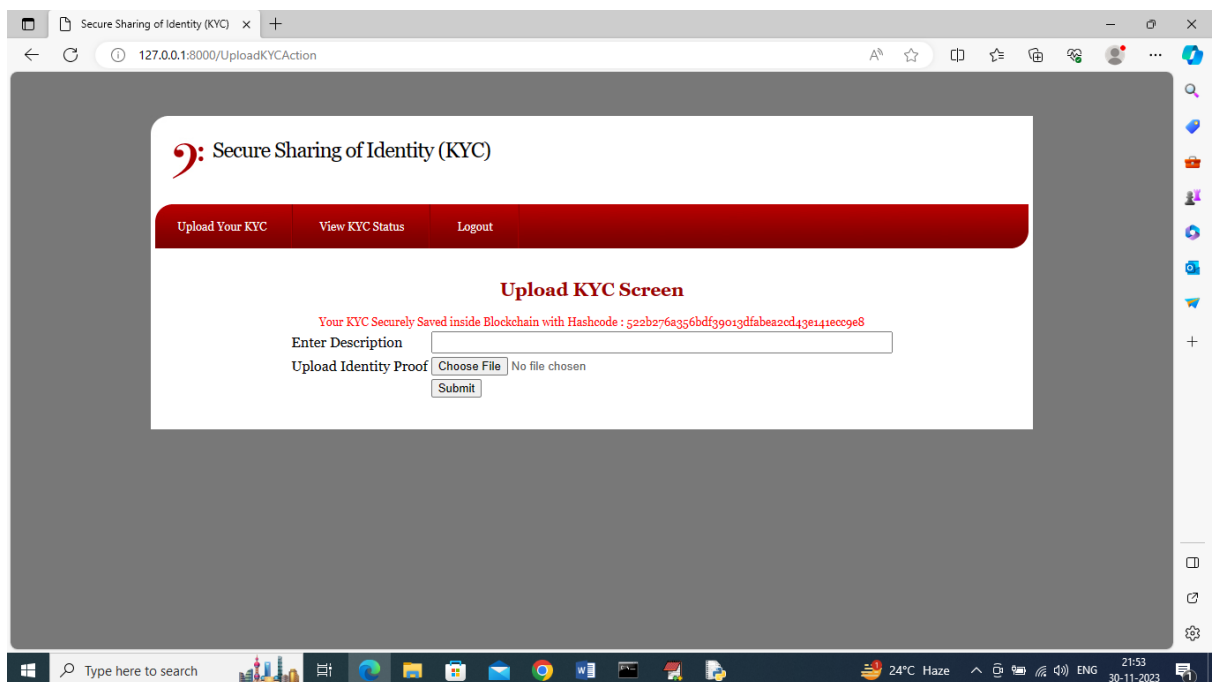
In above screen user is login and after login will get below page



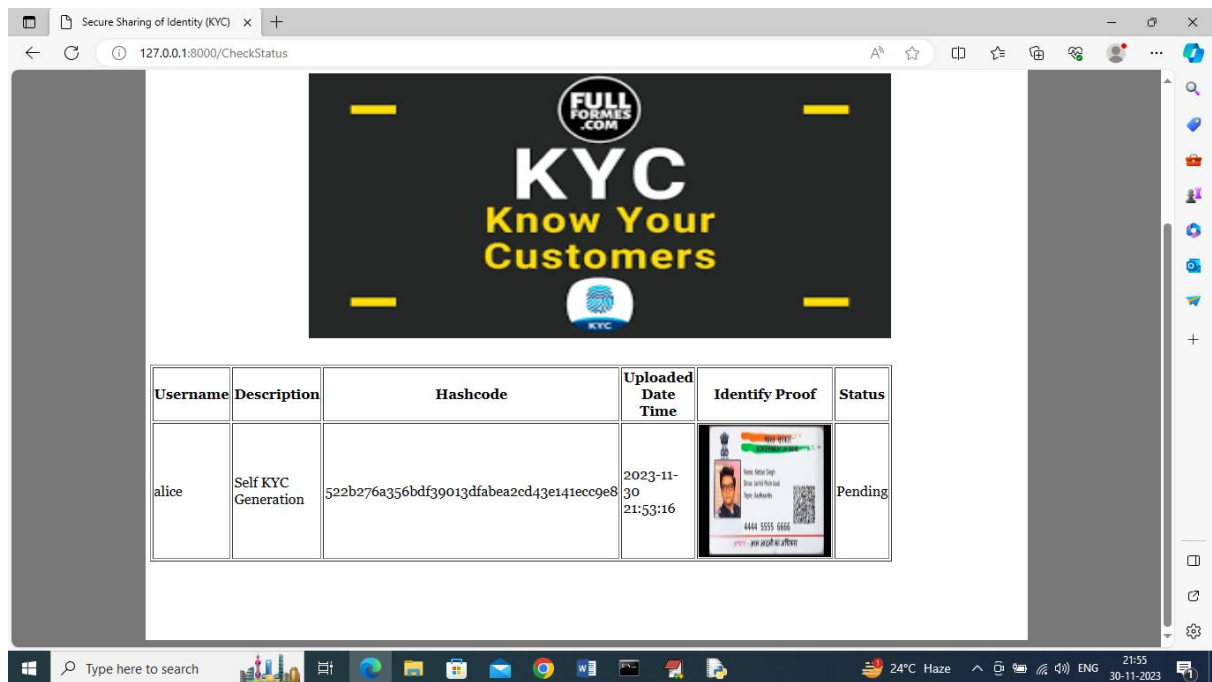
In above screen user can click on 'Upload Your KYC' link to upload KYC and get below page



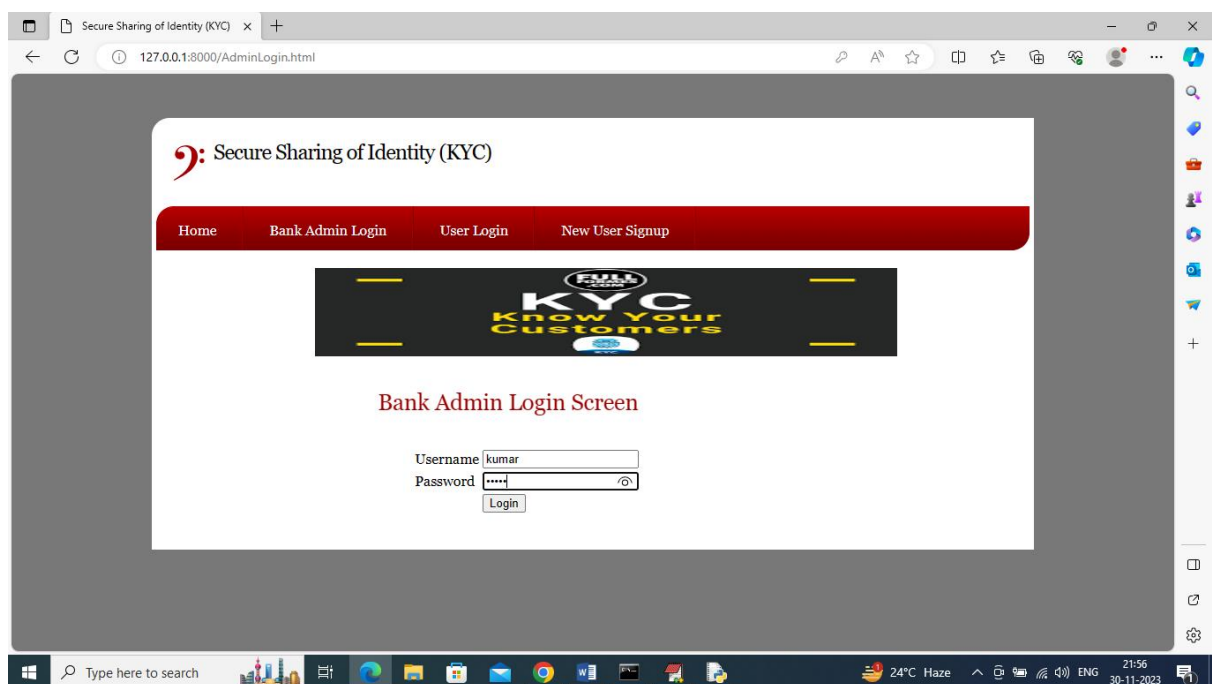
In above screen user can enter KYC details and then upload some type of identity proof and then press button to save KYC to Blockchain and get below page



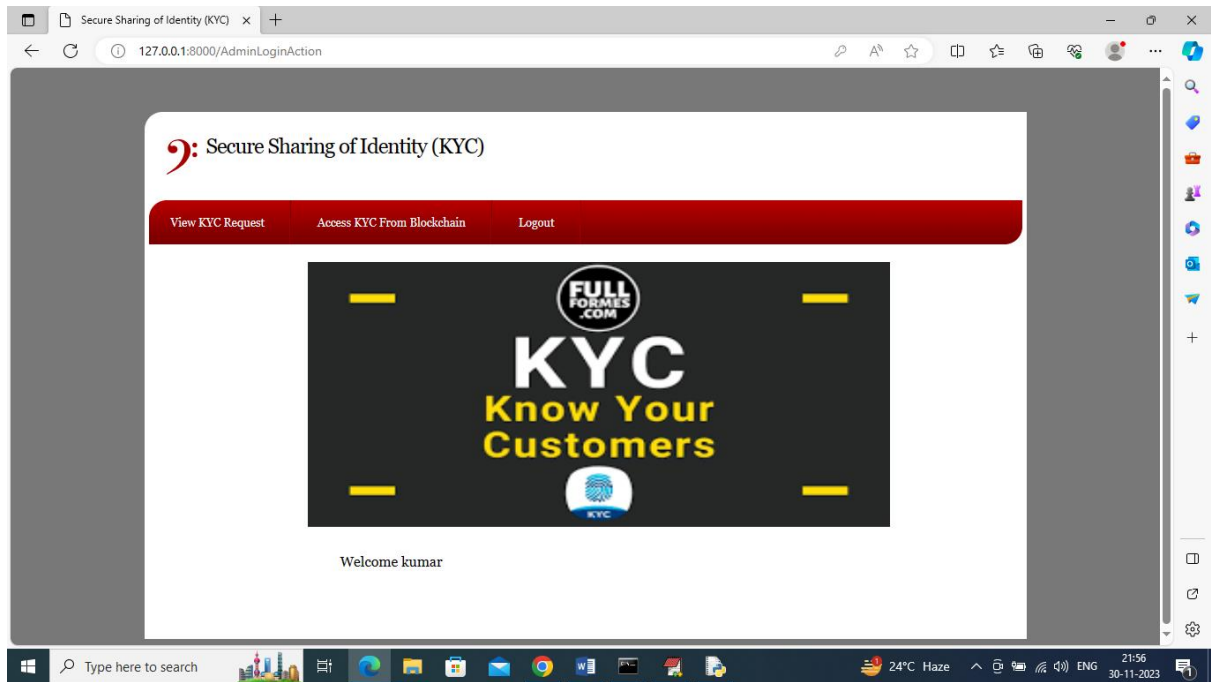
In above screen in red colour text can see KYC saved in Blockchain and displaying hash Blockchain address of KYC stored. Now KYC uploaded but not yet accepted by Bank Admin and now by click on 'View KYC Status' link to get below page



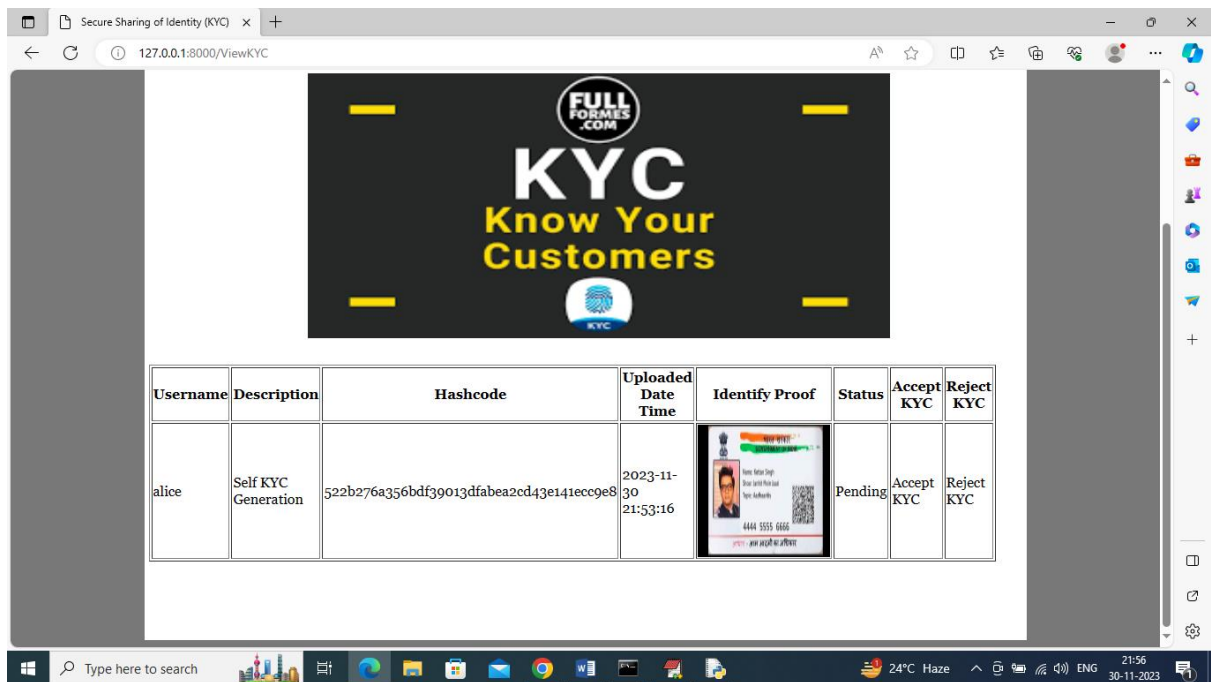
In above screen in last column user can see his KYC status as pending and now logout and login as 'Bank Admin' like below page



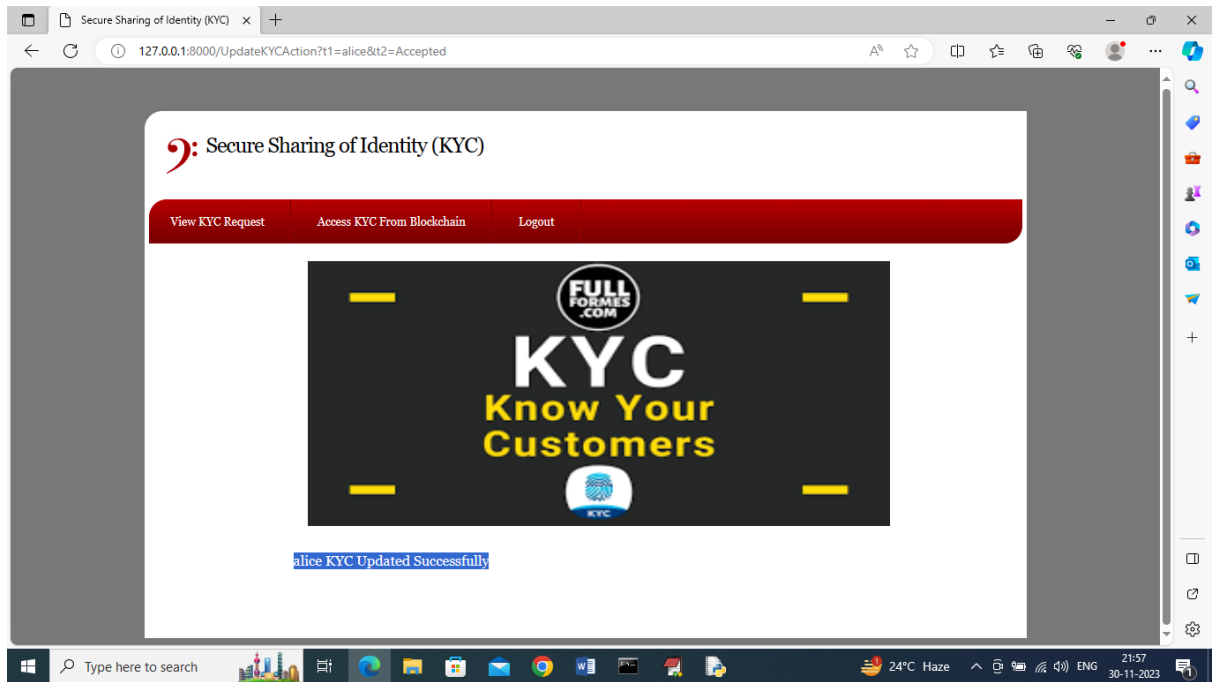
In above screen Bank Admin is login and after login will get below page



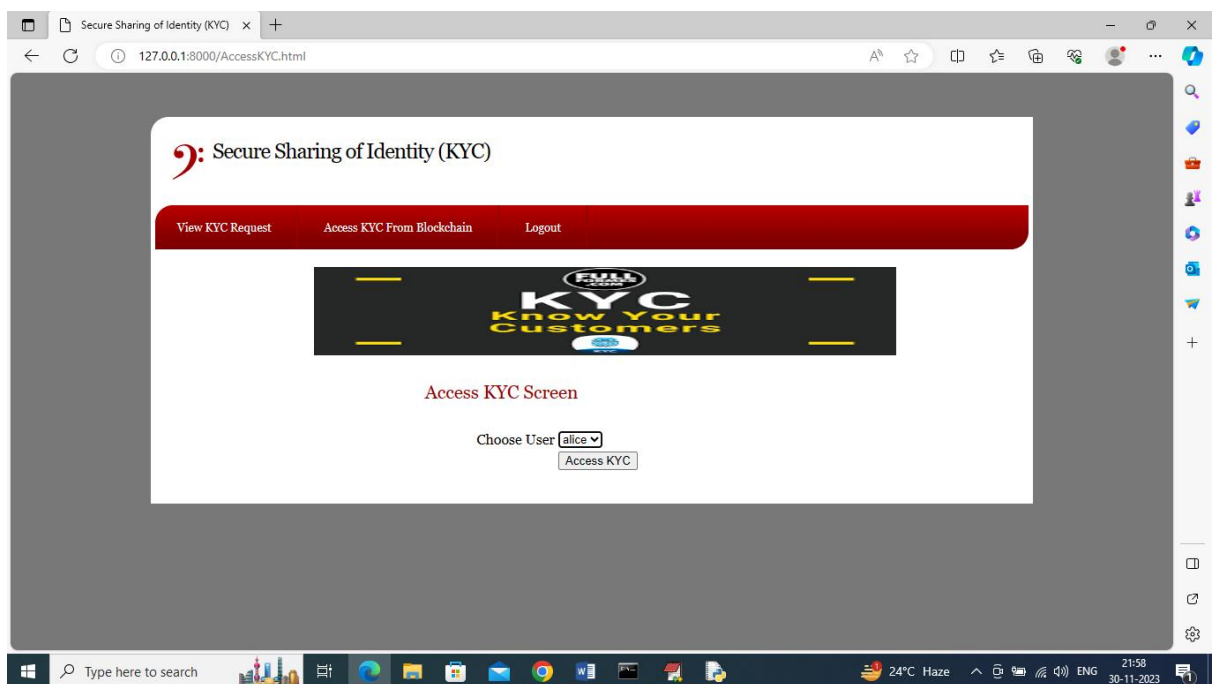
In above screen Bank Admin can click on ‘View KYC Request’ link to get below page



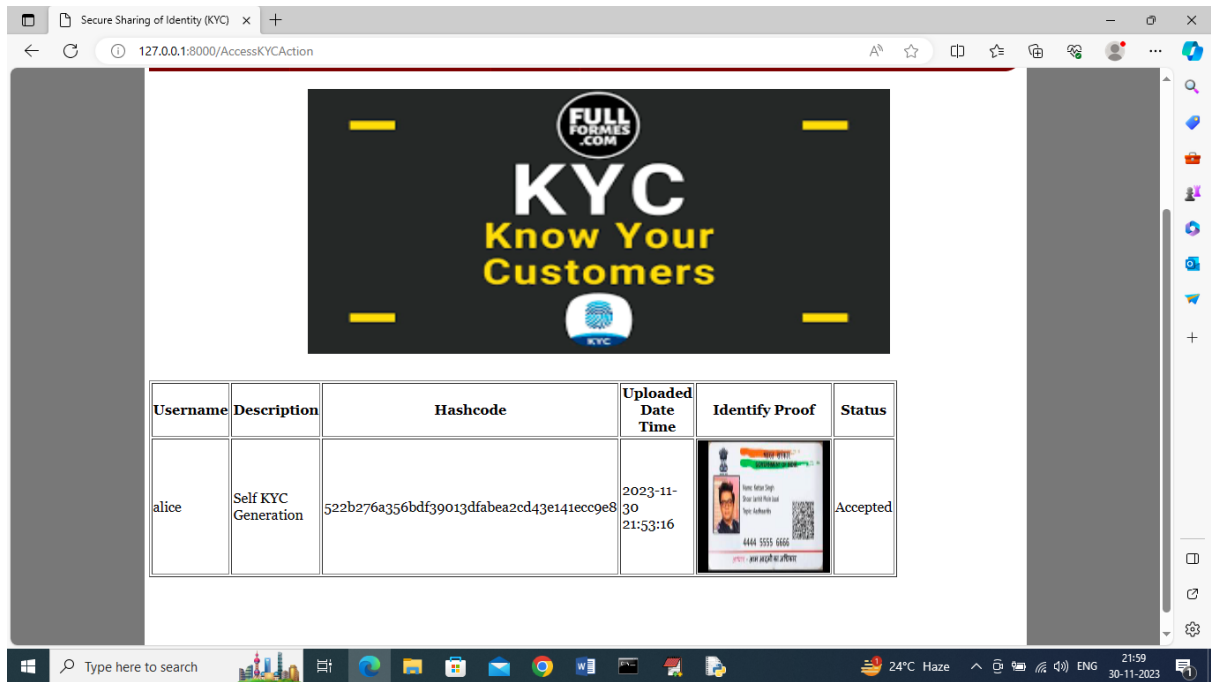
In above screen Bank Admin can see all details of user KYC and then click on either ‘Accept or Reject’ link to accept or reject KYC and get below page



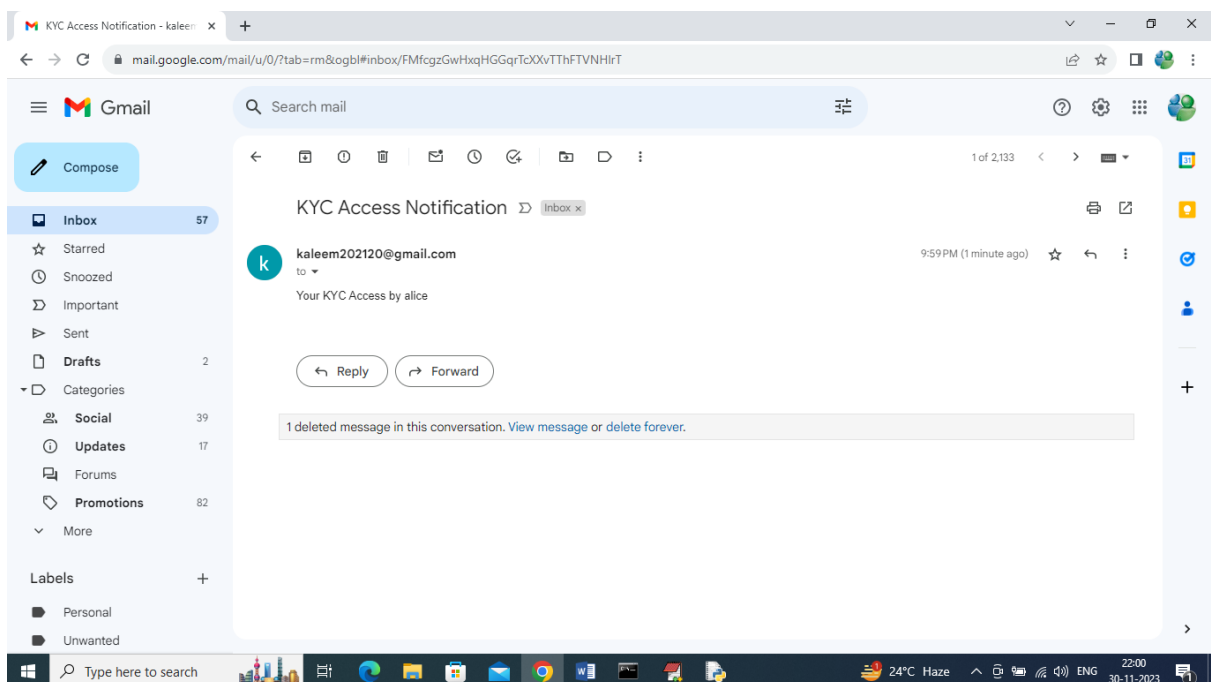
In above screen can see KYC accepted successfully and any Bank Admin can access that KYC by clicking on 'Access KYC from Blockchain' link to get below page



In above screen Bank Admin will select desired user and press button to access that KYC and get below output



In above screen Bank Admin Kumar can view above KYC with all details and once he accesses then Alice user will get notification in mail like below screen



In above mail screen can see Alice user got notification.

So by following above screens we can saved all users KYC in Blockchain securely and only those users can access this KYC who has access control to Blockchain and if any Bank Admin access user KYC then user will get Email notification

8. CONCLUSION

The "Secure Sharing of Identity (KYC)" project marks a paradigm shift in how identity information is managed and shared. By addressing the vulnerabilities in the existing system through the integration of blockchain technology and advanced notification mechanisms, the project strives to establish a secure, transparent, and user-centric approach to identity management. This innovation not only safeguards users against potential threats but also fosters trust in online identity verification processes. As we navigate the complexities of the digital age, the "Secure Sharing of Identity (KYC)" project stands as a pioneering effort to fortify the foundations of secure identity management in our interconnected world.

9. FUTURE ENHANCEMENT

The proposed system offers a secure and decentralized approach for KYC data sharing. However, future enhancements can significantly improve its efficiency, scalability, and usability. The following are potential areas for future development:

9.1 Integration with Government Digital ID Systems

- Link the KYC system with national identity platforms like **Aadhaar**, **Digi Locker**, or **e-Sign** for seamless verification.
- Real-time identity validation via authorized government APIs.

9.2 AI-based Fraud Detection

- Use **machine learning algorithms** to detect suspicious or fraudulent identity patterns.
- Anomaly detection for identifying repeated or fake KYC attempts.

9.3 Mobile Application Support

- Develop Android/iOS apps for better accessibility and real-time KYC management.
- Push notifications for KYC status updates, approvals, or rejections.

9.4 Zero-Knowledge Proofs (ZKP)

- Employ **ZKP** techniques to allow data verification without revealing the full document.
- Enhances **privacy** and **data minimization**.

9.5 Role-Based Access Control (RBAC)

- Implement fine-grained control for user roles such as Admin, Verifier, Customer.
- Allows different access levels for sensitive information.

9.6 Multi-language and Regional Support

- Add support for regional languages for broader user accessibility.
- Useful in multilingual countries like India.

9.7 Decentralized Identity (DID) Framework

- Align with **W3C DID standards** to allow users to own and control their digital identity without centralized control.

9.8 Biometric Verification

- Add **facial recognition, fingerprint, or iris scan** as part of the KYC process for additional security.

9.9 Data Analytics Dashboard

- Real-time analytics for regulators and businesses to view KYC trends, approval times, and compliance metrics.

9.10 Integration with Financial Institutions

- Direct API-based integration with banks, NBFCs, and fintech platforms for real-time KYC validation and onboarding.

10. REFERENCES

1. J. P. Moyano and O. Ross, "KYC optimization using distributed ledger technology", *Business & Information Systems Engineering*, vol. 59, no. 6, pp. 411-423, 2017.
2. N. K. Ostern and J. Riedel, "Know-Your-Customer (KYC) Requirements for Initial Coin Offerings", *Business & Information Systems Engineering*, pp. 1-17, 2020.
3. D. De Smet and A. L. Mention, "Improving auditor effectiveness in assessing KYC/AML practices: Case study in a Luxembourgish context", *Managerial Auditing Journal*, 2011.

4. R. Syah, M. K. Nasution, M. Elveny and H. Arbie, "Optimization model for customer behavior with MARS and KYC system", *Journal of Theoretical and Applied Information Technology*, vol. 98, no. 13, 2020.
5. I. Bashir, *Mastering Blockchain: Distributed ledger technology decentralization and smart contracts explained*, Packt Publishing Ltd, 2018.
6. G. Wood, "Ethereum: A secure decentralised generalised transaction ledger", *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1-32, 2014.
7. E. Androulaki et al., "Hyperledger fabric: a distributed operating system for permissioned blockchains", *Proceedings of the thirteenth EuroSys conference*, pp. 1-15, 2018.
8. W. Shbair, M. Steichen and J. François, "Blockchain orchestration and experimentation framework: A case study of KYC", *IEEE/IFIP Man2Block 2018-IEEE/IFIP Network Operations and Management Symposium*, 2018.
9. N. Sundareswaran, S. Sasirekha, I. J. L. Paul, S. Balakrishnan and G. Swaminathan, "Optimised KYC Blockchain System", *2020 International Conference on Innovative Trends in Information Technology (ICITIIT)*, pp. 1-6, 2020.
10. S. Nakamoto, "Re: Bitcoin P2P e-cash paper", *The Cryptography Mailing List*, 2008.
11. Y. Lootsma, "Blockchain as the newest regtech application—the opportunity to reduce the burden of kyc for financial institutions", *Banking & Financial Services Policy Report*, vol. 36, no. 8, pp. 16-21, 2017.
12. K. A. Lacey and B. C. George, "Crackdown on money laundering: a comparative analysis of the feasibility and effectiveness of domestic and multilateral policy reforms", *Nw. J. Int'l L. & Bus*, vol. 23, pp. 263, 2002.
13. R. J. Herring, "BCCI & Barings: Bank Resolutions Complicated by Fraud and Global Corporate Structure", *Systemic Financial Crises: Resolving Large Bank Insolvencies*, pp. 321-345, 2005.
14. D. Mingxiao, M. Xiaofeng, Z. Zhe, W. Xiangwei and C. Qijun, "A review on consensus algorithm of blockchain", *2017 IEEE international conference on systems man and cybernetics (SMC)*, pp. 2567-2572, 2017.

15. J. Sousa, A. Bessani and M. Vukolic, "A byzantine fault-tolerant ordering service for the hyperledger fabric blockchain platform", *2018 48th annual IEEE/IFIP international conference on dependable systems and networks (DSN)*, pp. 51-58, 2018.
16. S. A. Baset, L. Desrosiers, N. Gaur, P. Novotny, A. O'Dowd and V. Ramakrishna, *Hands-on blockchain with Hyperledger: building decentralized applications with Hyperledger Fabric and composer*, Packt Publishing Ltd, 2018.
17. C. Cachin, "Architecture of the hyperledger blockchain fabric", *Workshop on distributed cryptocurrencies and consensus ledgers*, vol. 310, no. 4, 2016.
18. P. Thakkar, S. Nathan and B. Viswanathan, "Performance benchmarking and optimizing hyperledger fabric blockchain platform", *2018 IEEE 26th International Symposium on Modeling Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pp. 264-276, 2018.
19. B. K. Mohanta, S. S. Panda and D. Jena, "An overview of smart contract and use cases in blockchain technology", *2018 9th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp. 1-4, 2018.
20. N. Atzei, M. Bartoletti and T. Cimoli, "A survey of attacks on ethereum smart contracts (sok)", *International conference on principles of security and trust*, pp. 164-186, 2017.
21. D. Puthal, N. Malik, S. P. Mohanty, E. Kougianos and C. Yang, "The blockchain as a decentralized security framework [future directions]", *IEEE Consumer Electronics Magazine*, vol. 7, no. 2, pp. 18-21, 2018.
22. V. Gramoli, "From blockchain consensus back to Byzantine consensus", *Future Generation Computer Systems*, vol. 107, pp. 760-769, 2020.